01	FILE	FILE 01 | main.js | 启动入口与云环境初始化
02	1	import { silenceConsoleInProduction } from '@/utils/logger.js';
03	2	import App from './App';
04	3	import AppBackgroundPageRoot from '@/components/AppBackgroundPageRoot.vue';
05	4	import cacheManager from '@/cache/core/manager.js';
06	5	import { getAppState, getOpenid } from '@/utils/app-state.js';
07	6	import { ensureRuntimeOpenid, ensureTcbAuthenticated, ensureTcbReady, installRuntimeBindings, se
08	↳	tupRuntimeSideEffects } from '@/utils/runtime-bootstrap.js';
09	7	import zpMixins from '@/uni_modules/zp-mixins/index.js';
10	8	import appFontManager from './utils/fontManager.js';
11	9	
12	10	silenceConsoleInProduction();
13	11	
14	12	function emitBuiltinFontLoaded() {
15	13	try {
16	14	uni.$emit && uni.$emit('font-loaded', { fontFamily: '汇文明朝' });
17	15	} catch (error) {}
18	16	}
19	17	
20	18	// #ifdef H5
21	19	if (typeof document !== 'undefined') {
22	20	const fontLink = document.createElement('link');
23	21	fontLink.rel = 'preload';
24	22	fontLink.as = 'font';
25	23	fontLink.type = 'font/woff2';
26	24	fontLink.href = '/static/fonts/Huiwen-mincho-compressed.woff2';
27	25	fontLink.crossOrigin = 'anonymous';
28	26	document.head.appendChild(fontLink);
29	27	
30	28	const fontStyle = document.createElement('style');
31	29	fontStyle.textContent = `
32	30	@font-face {
33	31	font-family: '汇文明朝';
34	32	src: url('/static/fonts/Huiwen-mincho-compressed.woff2') format('woff2');
35	33	font-weight: normal;
36	34	font-style: normal;
37	35	font-display: swap;
38	36	}
39	37	`;
40	38	document.head.appendChild(fontStyle);
41	39	
42	40	if (typeof FontFace !== 'undefined') {
43	41	const font = new FontFace('汇文明朝', 'url(/static/fonts/Huiwen-mincho-compressed.woff2)');
44	42	font.load().then((loadedFont) => {
45	43	document.fonts.add(loadedFont);
46	44	emitBuiltinFontLoaded();
47	45	}).catch((error) => {
48	46	console.warn('[font preload] h5 load failed', error);
49	47	});
50	48	}

```
01  49  }
02  50  // #endif
03  51
04  52  // #ifdef APP-PLUS
05  53  function preloadBuiltinAppFont() {
06  54    if (!appFontManager || typeof appFontManager.ensureFontAvailable !== 'function') {
07  55      return;
08  56    }
09  57    appFontManager.ensureFontAvailable('汇文明朝').then(() => {
10  58      emitBuiltinFontLoaded();
11  59    }).catch((error) => {
12  60      console.warn('[font preload] app plus load failed', error);
13  61    });
14  62  }
15  63
16  64  if (typeof plus !== 'undefined') {
17  65    preloadBuiltinAppFont();
18  66  }
19  67  // #endif
20  68
21  69  // #ifdef APP-HARMONY
22  70  console.log('[font preload] harmony uses bundled/static font fallback');
23  71  // #endif
24  72
25  73  // #ifdef MP-WEIXIN
26  74  console.log('[font preload] mp-weixin skips builtin preload');
27  75  // #endif
28  76
29  77  const runtimeTcb = ensureTcbReady();
30  78  setupRuntimeSideEffects();
31  79
32  80  try {
33  81    let debugEnabled = false;
34  82    try {
35  83      const value = uni.getStorageSync && uni.getStorageSync('CACHE_DEBUG');
36  84      debugEnabled = value === true || value === '1' || value === 1 || value === 'true';
37  85    } catch (error) {}
38  86    cacheManager.setDebug(!!debugEnabled);
39  87    if (typeof uni !== 'undefined') {
40  88      uni.$cacheStats = () => {
41  89        try {
42  90          console.log('[CacheStats]', cacheManager.getStats());
43  91        } catch (error) {
44  92          console.log('[CacheStats] failed', error);
45  93        }
46  94      };
47  95      uni.$cacheDebug = (enabled) => {
48  96        try {
49  97          cacheManager.setDebug(!!enabled);
50  98          console.log('[CacheDebug] =', !!enabled);
```

```
01  99      } catch (error) {}
02  100   };
03  101  }
04  102 } catch (error) {
05  103  console.warn('cache debug setup failed', error);
06  104 }
07  105
08  106 function normalizeCloudCallOptions(options = {}) {
09  107  const nextOptions = Object.assign({}, options);
10  108  delete nextOptions.__skipOpenidGuard;
11  109  return nextOptions;
12  110 }
13  111
14  112 async function resolveOpenidForCall(functionName) {
15  113  if (functionName === 'login' || functionName === 'getOpenId' || functionName === 'getPhoneNumb
16    ↳ erByToken') {
17  114    return { openid: null, allowed: true };
18  115  }
19  116
20  117  let openid = getOpenid();
21  118  if (!openid) {
22  119    openid = await ensureRuntimeOpenid();
23  120  }
24  121  if (!openid) {
25  122    const state = getAppState();
26  123    if (state._loginProcessCompleted && typeof uni !== 'undefined' && typeof uni.showToast === '
27    ↳ function') {
28  124      uni.showToast({ title: '用户未登录', icon: 'none' });
29  125    }
30  126    return { openid: null, allowed: false };
31  127  }
32  128
33  129  return { openid, allowed: true };
34  130 }
35  131
36  132 // #ifdef APP-PLUS || APP-HARMONY
37  133 if (typeof uniCloud !== 'undefined' && typeof uniCloud.callFunction === 'function') {
38  134  const originalUniCloudCallFunction = uniCloud.callFunction.bind(uniCloud);
39  135  uniCloud.callFunction = async function(options = {}) {
40  136    const normalizedOptions = normalizeCloudCallOptions(options);
41  137    const name = normalizedOptions.name;
42  138    if (!name || options.__skipOpenidGuard) {
43  139      return originalUniCloudCallFunction(normalizedOptions);
44  140    }
45  141
46  142    const { openid, allowed } = await resolveOpenidForCall(name);
47  143    if (!allowed) {
48  144      const error = new Error('NO_OPENID');
49  145      error.code = 'NO_OPENID';
50  146      if (typeof normalizedOptions.fail === 'function') {
```

```
01  147         normalizedOptions.fail(error);
02  148       }
03  149       if (typeof normalizedOptions.complete === 'function') {
04  150         normalizedOptions.complete(error);
05  151       }
06  152       throw error;
07  153     }
08  154
09  155     const data = Object.assign({}, normalizedOptions.data || {});
10  156     if (openid && !data.openid) {
11  157       data.openid = openid;
12  158     }
13  159     return originalUniCloudCallFunction(Object.assign({}, normalizedOptions, { data }));
14  160   };
15  161 }
16  162 // #endif
17  163
18  164 // #ifdef H5 || APP-PLUS || APP-HARMONY
19  165 if (runtimeTcb && typeof runtimeTcb.callFunction === 'function') {
20  166   const originalTcbCallFunction = runtimeTcb.callFunction.bind(runtimeTcb);
21  167   runtimeTcb.callFunction = async function(options = {}) {
22  168     const normalizedOptions = normalizeCloudCallOptions(options);
23  169     const name = normalizedOptions.name;
24  170     if (!name || options.__skipOpenidGuard) {
25  171       return originalTcbCallFunction(normalizedOptions);
26  172     }
27  173
28  174     await ensureTcbAuthenticated(runtimeTcb);
29  175
30  176     const { openid, allowed } = await resolveOpenidForCall(name);
31  177     if (!allowed) {
32  178       const error = new Error('NO_OPENID');
33  179       error.code = 'NO_OPENID';
34  180       if (typeof normalizedOptions.fail === 'function') {
35  181         normalizedOptions.fail(error);
36  182       }
37  183       if (typeof normalizedOptions.complete === 'function') {
38  184         normalizedOptions.complete(error);
39  185       }
40  186       throw error;
41  187     }
42  188
43  189     const data = Object.assign({}, normalizedOptions.data || {});
44  190     if (openid && !data.openid) {
45  191       data.openid = openid;
46  192     }
47  193     return originalTcbCallFunction(Object.assign({}, normalizedOptions, { data }));
48  194   };
49  195 }
50  196 // #endif
```

01	197	
02	198	function notifyH5AppReady() {
03	199	// #ifdef H5
04	200	if (typeof window === 'undefined') {
05	201	return;
06	202	}
07	203	if (typeof window.__ENTERAPP_HIDE_BOOT_MASK__ === 'function') {
08	204	window.__ENTERAPP_HIDE_BOOT_MASK__();
09	205	}
10	206	window.dispatchEvent(new Event('enterapp-ready'));
11	207	// #endif
12	208	}
13	209	
14	210	import { createSSRApp } from 'vue';
15	211	
16	212	export function createApp() {
17	213	const app = createSSRApp(App);
18	214	installRuntimeBindings(app);
19	215	app.use(zpMixins);
20	216	app.component('app-background-page-root', AppBackgroundPageRoot);
21	217	notifyH5AppReady();
22	218	return {
23	219	app
24	220	};
25	221	}
26	**FILE**	**FILE 02 | App.vue |** 应用级生命周期与全局逻辑
27	1	<script>
28	2	// #ifdef APP-PLUS
29	3	import { checkAndUpdate } from '@/utils/hotUpdate.js';
30	4	// #endif
31	5	import {
32	6	getAppState,
33	7	markLoginProcessCompleted,
34	8	markLoginProcessStarted,
35	9	patchAppState,
36	10	setUserSession
37	11	} from '@/utils/app-state.js';
38	12	import { formatErrorForLog } from '@/utils/error-log.js';
39	13	import { ensureRuntimeOpenid, ensureTcbAuthenticated, installRuntimeBindings } from '@/utils/run
40	↳	time-bootstrap.js';
41	14	
42	15	export default {
43	16	data() {
44	17	return {
45	18	appState: {
46	19	userInfo: null,
47	20	openid: null,
48	21	isLoggedIn: false,
49	22	_loginProcessStarted: false,
50	23	_loginProcessCompleted: false

```
01  24          }
02  25      };
03  26  },
04  27
05  28  onLaunch() {
06  29      installRuntimeBindings(this);
07  30      this.syncGlobalDataFromAppState();
08  31      // #ifdef APP-PLUS
09  32      this.runWhenPlusReady((plusInstance) => {
10  33          const args = this.getPlusRuntimeArguments(plusInstance);
11  34          if (args && args.includes('github-callback')) {
12  35              this.handleUrlScheme({ path: args });
13  36          }
14  37      }, 'handle launch args');
15  38      // #endif
16  39
17  40      uni.removeStorageSync('cachedPostList');
18  41
19  42      // #ifdef APP-PLUS
20  43      this.runWhenPlusReady((plusInstance) => {
21  44          try {
22  45              plusInstance.runtime.getProperty(plusInstance.runtime.appid, () => {
23  46                  checkAndUpdate({ silent: true, showConfirm: true }).catch((error) => {
24  47                      const message = error && (error.errMsg || error.message || String(error)
25  ↳ );
26  48                      if (!message || (!message.includes('resource exhausted') && !message.inc
27  ↳ ludes('ResourceExhausted'))) {
28  49                          console.warn(`[App] hot update check skipped: ${formatErrorForLog(er
29  ↳ ror)}`);
30  50                      }
31  51                  });
32  52              });
33  53          } catch (error) {
34  54              console.warn(`[App] hot update bootstrap failed: ${formatErrorForLog(error)}`);
35  55          }
36  56      }, 'check hot update');
37  57      // #endif
38  58
39  59      markLoginProcessStarted();
40  60      this.syncGlobalDataFromAppState();
41  61
42  62      // #ifdef MP-WEIXIN
43  63      if (!this.appState._mpBuiltinFontPreloadStarted) {
44  64          this.applyAppState({ _mpBuiltinFontPreloadStarted: true });
45  65          try { uni.setStorageSync('__builtin_font_huiwen_ready__', false); } catch (_) {}
46  66      }
47  67      // #endif
48  68
49  69      this.$nextTick(() => {
50  70          this.loginAndCheckUser();
```

```
01  71          });
02  72      },
03  73
04  74      onShow() {
05  75          // #ifdef APP-PLUS
06  76          this.runWhenPlusReady((plusInstance) => {
07  77              const args = this.getPlusRuntimeArguments(plusInstance);
08  78              if (args && args.includes('github-callback')) {
09  79                  this.handleUrlScheme({ path: args });
10  80              }
11  81
12  82              if (!this._newintentListenerAdded && plusInstance.globalEvent && plusInstance.global
13  ↳ Event.addEventListener) {
14  83                  plusInstance.globalEvent.addEventListener('newintent', () => {
15  84                      const nextArgs = this.getPlusRuntimeArguments(plusInstance);
16  85                      if (nextArgs) {
17  86                          this.handleUrlScheme({ path: nextArgs });
18  87                      }
19  88                  });
20  89                  this._newintentListenerAdded = true;
21  90              }
22  91          }, 'handle app show');
23  92          // #endif
24  93      },
25  94
26  95      methods: {
27  96          syncGlobalDataFromAppState() {
28  97              this.appState = Object.assign({}, this.appState, getAppState());
29  98          },
30  99
31 100          applyAppState(partial = {}) {
32 101              patchAppState(partial);
33 102              this.syncGlobalDataFromAppState();
34 103          },
35 104
36 105          applyUserSession(userInfo, openid = null, extra = {}) {
37 106              setUserSession(userInfo, openid);
38 107              if (extra && Object.keys(extra).length > 0) {
39 108                  patchAppState(extra);
40 109              }
41 110              this.syncGlobalDataFromAppState();
42 111          },
43 112
44 113          finishLoginProcess(extra = {}) {
45 114              markLoginProcessCompleted();
46 115              if (extra && Object.keys(extra).length > 0) {
47 116                  patchAppState(extra);
48 117              }
49 118              this.syncGlobalDataFromAppState();
50 119          },
```

```
01  120
02  121         runWhenPlusReady(handler, taskLabel) {
03  122             // #ifdef APP-PLUS
04  123             const execute = (plusInstance) => {
05  124                 try {
06  125                     handler && handler(plusInstance);
07  126                 } catch (error) {
08  127                     console.warn(`[App] ${taskLabel || 'app-plus task'} failed: ${formatErrorFor
09  ↳ Log(error)}`);
10  128                 }
11  129             };
12  130
13  131             if (typeof plus !== 'undefined' && plus && plus.runtime) {
14  132                 execute(plus);
15  133                 return;
16  134             }
17  135
18  136             const start = Date.now();
19  137             const timer = setInterval(() => {
20  138                 if (typeof plus !== 'undefined' && plus && plus.runtime) {
21  139                     clearInterval(timer);
22  140                     execute(plus);
23  141                     return;
24  142                 }
25  143
26  144                 if (Date.now() - start >= 10000) {
27  145                     clearInterval(timer);
28  146                     console.warn(`[App] ${taskLabel || 'app-plus task'} skipped: plus not ready`
29  ↳ );
30  147                 }
31  148             }, 50);
32  149             // #endif
33  150         },
34  151
35  152         getPlusRuntimeArguments(plusInstance) {
36  153             try {
37  154                 return plusInstance && plusInstance.runtime ? (plusInstance.runtime.arguments ||
38  ↳  '') : '';
39  155             } catch (error) {
40  156                 console.warn(`[App] read plus runtime arguments failed: ${formatErrorForLog(erro
41  ↳ r)}`);
42  157                 return '';
43  158             }
44  159         },
45  160
46  161         handleUrlScheme(options) {
47  162             // #ifdef APP-PLUS
48  163             if (!(options && options.path) || !options.path.includes('github-callback')) {
49  164                 return;
50  165             }
```

```
01  166
02  167             try {
03  168                 const params = {};
04  169                 const queryString = options.path.split('?')[1];
05  170                 if (queryString) {
06  171                     queryString.split('&').forEach((pair) => {
07  172                         const [key, value] = pair.split('=');
08  173                         params[key] = decodeURIComponent(value || '');
09  174                     });
10  175                 }
11  176
12  177                 const type = params.type;
13  178                 const data = params.data;
14  179
15  180                 if (type === 'register') {
16  181                     try {
17  182                         uni.setStorageSync('github_temp_data', JSON.parse(data));
18  183                     } catch (error) {
19  184                         console.warn(`[App] parse github register payload failed: ${formatErrorF
20    ↳ orLog(error)}`);
21  185                     }
22  186
23  187                     uni.showToast({ title: '欢迎! 请完成注册', icon: 'none', duration: 2000 });
24  188                     setTimeout(() => {
25  189                         uni.reLaunch({
26  190                             url: `/pages/register/register?fromGithub=true&githubData=${encodeUR
27    ↳ IComponent(data)}`
28  191                         });
29  192                     }, 500);
30  193                     return;
31  194                 }
32  195
33  196                 if (type === 'login') {
34  197                     const loginData = JSON.parse(data || '{}');
35  198                     const userInfo = loginData.user;
36  199
37  200                     this.applyUserSession(userInfo, userInfo && (userInfo._openid || userInfo.op
38    ↳ enid), {
39  201                         _loginProcessCompleted: true,
40  202                         isLoggedIn: true
41  203                     });
42  204
43  205                     uni.setStorageSync('userInfo', userInfo);
44  206                     uni.setStorageSync('github_access_token', loginData.accessToken);
45  207
46  208                     uni.showToast({ title: '登录成功! ', icon: 'success', duration: 2000 });
47  209                     setTimeout(() => {
48  210                         uni.reLaunch({ url: '/pages/poem-square/poem-square' });
49  211                     }, 500);
50  212                     return;
```

```
01  213                 }
02  214
03  215                 if (type === 'error') {
04  216                     const errorInfo = JSON.parse(data || '{}');
05  217                     uni.showModal({
06  218                         title: 'GitHub 登录失败',
07  219                         content: errorInfo.message || 'GitHub 登录失败，请重试',
08  220                         showCancel: false,
09  221                         confirmText: '知道了',
10  222                         success: () => {
11  223                             uni.reLaunch({ url: '/pages/login/login' });
12  224                         }
13  225                     });
14  226                 }
15  227             } catch (error) {
16  228                 console.error(`[App] handle github callback failed: ${formatErrorForLog(error)}`
17    ↳ );
18  229                 uni.showToast({
19  230                     title: '授权失败，请重试',
20  231                     icon: 'none',
21  232                     duration: 2000
22  233                 });
23  234             }
24  235             // #endif
25  236         },
26  237
27  238         getCurrentPagePath() {
28  239             try {
29  240                 const pages = getCurrentPages();
30  241                 const currentPage = pages[pages.length - 1];
31  242                 if (currentPage) {
32  243                     return currentPage.route || currentPage.$page?.fullPath || '';
33  244                 }
34  245             } catch (error) {
35  246                 console.warn(`[App] get current page failed: ${formatErrorForLog(error)}`);
36  247             }
37  248             return '';
38  249         },
39  250
40  251         isAllowedPageForUnauthenticated() {
41  252             return true;
42  253         },
43  254
44  255         async loginAndCheckUser() {
45  256             installRuntimeBindings(this);
46  257
47  258             if (!this.$tcb) {
48  259                 console.error('[App] runtime tcb unavailable');
49  260                 this.finishLoginProcess();
50  261                 return;
```

```
01  262             }
02  263
03  264             try {
04  265                 await ensureTcbAuthenticated(this.$tcb);
05  266                 const bootstrappedOpenid = await ensureRuntimeOpenid();
06  267                 if (bootstrappedOpenid) {
07  268                     this.applyAppState({ openid: bootstrappedOpenid });
08  269                 }
09  270             } catch (error) {
10  271                 console.warn(`[App] bootstrap openid failed: ${formatErrorForLog(error)}`);
11  272             }
12  273
13  274             const cachedUserInfo = uni.getStorageSync('userInfo');
14  275             if (cachedUserInfo && cachedUserInfo._openid) {
15  276                 try {
16  277                     await ensureTcbAuthenticated(this.$tcb);
17  278
18  279                     const verifyRes = await this.$tcb.callFunction({
19  280                         name: 'getUserProfile',
20  281                         data: { userId: cachedUserInfo._openid }
21  282                     });
22  283
23  284                     if (verifyRes.result && verifyRes.result.success && verifyRes.result.userInf
24  ↳ o) {
25  285                         const latestUserInfo = verifyRes.result.userInfo;
26  286                         this.applyUserSession(latestUserInfo, latestUserInfo._openid, {
27  287                             _loginProcessCompleted: true,
28  288                             isLoggedIn: true
29  289                         });
30  290                         uni.setStorageSync('userInfo', latestUserInfo);
31  291                         return;
32  292                     }
33  293
34  294                     uni.removeStorageSync('userInfo');
35  295                 } catch (error) {
36  296                     console.error(`[App] verify cached user failed: ${formatErrorForLog(error)}`
37  ↳ );
38  297                     uni.removeStorageSync('userInfo');
39  298                 }
40  299             }
41  300
42  301             try {
43  302                 await ensureTcbAuthenticated(this.$tcb);
44  303
45  304                 let openid = this.appState.openid;
46  305                 if (!openid) {
47  306                     const loginRes = await this.$tcb.callFunction({
48  307                         name: 'login',
49  308                         __skipOpenidGuard: true
50  309                     });
```

```
01  310
02  311                     if (loginRes.result && loginRes.result.openid) {
03  312                         openid = loginRes.result.openid;
04  313                     } else if (loginRes.openid) {
05  314                         openid = loginRes.openid;
06  315                     } else if (loginRes.result && loginRes.result.uid) {
07  316                         openid = loginRes.result.uid;
08  317                     }
09  318                 }
10  319
11  320                 if (!openid) {
12  321                     throw new Error('云函数 login 未返回 openid');
13  322                 }
14  323
15  324                 this.applyAppState({ openid });
16  325                 uni.setStorageSync('userOpenId', openid);
17  326
18  327                 const db = this.$tcb.database();
19  328                 const userRes = await db.collection('users').where({ _openid: openid }).get();
20  329
21  330                 if (Array.isArray(userRes.data) && userRes.data.length > 0) {
22  331                     const userInfo = userRes.data[0];
23  332                     this.applyAppState({ userInfo });
24  333                     uni.setStorageSync('userInfo', userInfo);
25  334                 } else {
26  335                     this.applyAppState({ userInfo: null });
27  336                 }
28  337
29  338                 this.finishLoginProcess({
30  339                     userInfo: this.appState.userInfo,
31  340                     openid: this.appState.openid,
32  341                     isLoggedIn: !!this.appState.userInfo
33  342                 });
34  343             } catch (error) {
35  344                 console.error(`[App] login bootstrap failed: ${formatErrorForLog(error)}`);
36  345                 this.finishLoginProcess();
37  346                 uni.showToast({
38  347                     icon: 'none',
39  348                     title: '登录失败，请稍后重试'
40  349                 });
41  350             }
42  351         }
43  352     }
44  353 };
45  354 </script>
46  355
47  356 <style>
48  357 /* 全局字体预加载 - 仅用于诗歌内容 */
49  358 /* 小程序不支持 CSS @font-face 加载本地字体，只在 H5 和 APP 环境使用 */
50  359 /* #ifndef MP-WEIXIN */
```

```
@font-face {
  font-family: '汇文明朝';
  src: url('/static/fonts/Huiwen-mincho-compressed.woff2') format('woff2');
  font-weight: normal;
  font-style: normal;
  font-display: swap;
}
/* #endif */

/* 全局样式：修改下拉刷新的loading转圈圈颜色为黑色 */
/* #ifdef MP-WEIXIN */
.wx-pull-refresh {
    color: #000000 !important;
}

.wx-pull-refresh .wx-pull-refresh-spinner {
    color: #000000 !important;
}

.wx-pull-refresh .wx-pull-refresh-spinner::before {
    color: #000000 !important;
}
/* #endif */

/* #ifdef H5 */
.uni-pull-refresh {
    color: #000000 !important;
}

.uni-pull-refresh .uni-pull-refresh-spinner {
    color: #000000 !important;
}

.uni-pull-refresh .uni-pull-refresh-spinner::before {
    color: #000000 !important;
}
/* #endif */

/* #ifdef APP-PLUS */
.uni-pull-refresh {
    color: #000000 !important;
}

.uni-pull-refresh .uni-pull-refresh-spinner {
    color: #000000 !important;
}

.uni-pull-refresh .uni-pull-refresh-spinner::before {
    color: #000000 !important;
}
```

01	410	/* #endif */
02	411	
03	412	.uni-pull-refresh,
04	413	.wx-pull-refresh,
05	414	.pull-refresh {
06	415	color: #000000 !important;
07	416	}
08	417	
09	418	.uni-pull-refresh .uni-pull-refresh-spinner,
10	419	.wx-pull-refresh .wx-pull-refresh-spinner,
11	420	.pull-refresh .pull-refresh-spinner {
12	421	color: #000000 !important;
13	422	border-color: #000000 !important;
14	423	}
15	424	
16	425	.uni-pull-refresh .uni-pull-refresh-spinner::before,
17	426	.wx-pull-refresh .wx-pull-refresh-spinner::before,
18	427	.pull-refresh .pull-refresh-spinner::before {
19	428	color: #000000 !important;
20	429	border-color: #000000 !important;
21	430	}
22	431	
23	432	.uni-pull-refresh-indicator,
24	433	.wx-pull-refresh-indicator {
25	434	color: #000000 !important;
26	435	}
27	436	
28	437	.uni-pull-refresh-indicator .uni-pull-refresh-spinner,
29	438	.wx-pull-refresh-indicator .wx-pull-refresh-spinner {
30	439	color: #000000 !important;
31	440	border-color: #000000 !important;
32	441	}
33	442	</style>
34	**FILE**	**FILE 03 | utils/cloudCall.js |** 统一云函数调用封装
35	1	import { getCloudFunctionMethod } from './platformDetector.js';
36	2	import auth from './auth.js';
37	3	import { formatErrorForLog } from './error-log.js';
38	4	
39	5	const platformDetector = {
40	6	getCloudFunctionMethod
41	7	};
42	8	
43	9	const DEFAULT_RETRY_DELAY = 300;
44	10	
45	11	function getCachedAppInstance() {
46	12	if (typeof uni === 'undefined') {
47	13	return null;
48	14	}
49	15	return uni.$appInstance || null;
50	16	}

```
01  17
02  18  function ensureObject(value) {
03  19      if (!value || typeof value !== 'object') {
04  20          return {};
05  21      }
06  22      if (Array.isArray(value)) {
07  23          return value.slice();
08  24      }
09  25      return Object.assign({}, value);
10  26  }
11  27
12  28  function createError(code, message, originalError) {
13  29      const error = originalError instanceof Error ? originalError : new Error(message || code);
14  30      error.code = code;
15  31      if (originalError && originalError !== error) {
16  32          error.originalError = originalError;
17  33      }
18  34      return error;
19  35  }
20  36
21  37  function getTcbInstance(context) {
22  38      if (context && context.$tcb && typeof context.$tcb.callFunction === 'function') {
23  39          return context.$tcb;
24  40      }
25  41      const app = getCachedAppInstance();
26  42      if (app && app.$tcb && typeof app.$tcb.callFunction === 'function') {
27  43          return app.$tcb;
28  44      }
29  45      if (typeof uni !== 'undefined' && uni.$tcb && typeof uni.$tcb.callFunction === 'function') {
30  46          return uni.$tcb;
31  47      }
32  48      return null;
33  49  }
34  50
35  51  function getTcbAuthEnsurer(context) {
36  52      if (context && typeof context.$ensureTcbAuthenticated === 'function') {
37  53          return context.$ensureTcbAuthenticated;
38  54      }
39  55      const app = getCachedAppInstance();
40  56      if (app && typeof app.$ensureTcbAuthenticated === 'function') {
41  57          return app.$ensureTcbAuthenticated;
42  58      }
43  59      if (typeof uni !== 'undefined' && typeof uni.$ensureTcbAuthenticated === 'function') {
44  60          return uni.$ensureTcbAuthenticated;
45  61      }
46  62      return null;
47  63  }
48  64
49  65  async function ensureTcbCallReady(instance, context) {
50  66      if (!instance || instance.__skipAuth || typeof instance.auth !== 'function') {
```

```
01  67      return instance;
02  68  }
03  69
04  70  const authEnsurer = getTcbAuthEnsurer(context);
05  71  if (typeof authEnsurer === 'function') {
06  72      await authEnsurer(instance);
07  73      return instance;
08  74  }
09  75
10  76  const authClient = instance.auth();
11  77  if (!authClient || authClient.currentUser || typeof authClient.signInAnonymously !== 'functi
12  ↳ on') {
13  78      return instance;
14  79  }
15  80
16  81  await authClient.signInAnonymously();
17  82  return instance;
18  83  }
19  84
20  85  async function invokeCloudFunction(method, payload) {
21  86  console.log(`[invokeCloudFunction] method: ${method}`);
22  87
23  88  if (method === 'tcb') {
24  89      const instance = getTcbInstance(payload.context);
25  90      console.log('[invokeCloudFunction] TCB instance:', instance ? 'available' : 'missing');
26  91      if (!instance) {
27  92          throw createError('TCB_NOT_AVAILABLE', 'TCB instance unavailable');
28  93      }
29  94      try {
30  95          await ensureTcbCallReady(instance, payload.context);
31  96      } catch (error) {
32  97          throw createError('TCB_AUTH_FAILED', 'TCB auth unavailable', error);
33  98      }
34  99      return instance.callFunction(payload.options, undefined, payload.customReqOpts);
35  100 }
36  101
37  102 if (method === 'wx-cloud') {
38  103     console.log('[invokeCloudFunction] using wx.cloud');
39  104     if (typeof wx === 'undefined' || !wx.cloud) {
40  105         throw createError('WX_CLOUD_NOT_AVAILABLE', 'wx.cloud unavailable');
41  106     }
42  107
43  108     console.log('[invokeCloudFunction] wx.cloud function:', payload.options.name);
44  109     return wx.cloud.callFunction(payload.options);
45  110 }
46  111
47  112 if (typeof uniCloud !== 'undefined' && typeof uniCloud.callFunction === 'function') {
48  113     return uniCloud.callFunction(payload.options);
49  114 }
50  115
```

```
    throw createError('NO_CLOUD_METHOD', 'No supported cloud function method');
}

export async function cloudCall(name, data = {}, options = {}) {
    if (!name || typeof name !== 'string') {
        return Promise.reject(createError('INVALID_NAME', 'Invalid cloud function name'));
    }

    const {
        pageTag = 'global',
        retry = 0,
        retryDelay = DEFAULT_RETRY_DELAY,
        context,
        injectOpenId,
        requireAuth = false,
        timeoutMs
    } = options;
    const shouldInjectOpenId = typeof injectOpenId === 'boolean' ? injectOpenId : !['login', 'ge
↳ tOpenId', 'github-auth'].includes(name);

    const payload = ensureObject(data);
    let openid = null;

    if (shouldInjectOpenId) {
        try {
            openid = await auth.getOpenId();
        } catch (error) {
            console.error(`[cloudCall][${pageTag}] failed to get openid: ${formatErrorForLog(err
↳ or)}`);
            throw createError('NO_OPENID', '\u83B7\u53D6 openid \u5931\u8D25', error);
        }

        if (!openid) {
            if (requireAuth) {
                const error = createError('NO_OPENID', '\u7528\u6237\u672A\u767B\u5F55\u6216 ope
↳ nid \u7F3A\u5931');

                if (typeof uni !== 'undefined') {
                    try {
                        const authHelper = await import('./authHelper.js');
                        await authHelper.requireLogin({
                            content: '\u6B64\u64CD\u4F5C\u9700\u8981\u767B\u5F55\uFF0C\u8BF7\u51
↳ 48\u767B\u5F55'
                        });
                    } catch (importError) {
                        if (uni.showToast) {
                            uni.showToast({
                                title: '\u8BF7\u5148\u767B\u5F55',
                                icon: 'none'
                            });
```

```
01  162                     }
02  163                 }
03  164             }
04  165
05  166             console.warn(`[cloudCall][${pageTag}] openid missing, "${name}" requires auth`);
06  167             throw error;
07  168         }
08  169         console.warn(`[cloudCall][${pageTag}] openid missing, "${name}" will run without inj
09      ↳ ection`);
10  170     } else if (!payload.openid) {
11  171         payload.openid = openid;
12  172     }
13  173 }
14  174
15  175 const method = platformDetector.getCloudFunctionMethod();
16  176 console.log(`[cloudCall] method=${method}, name=${name}`);
17  177 const totalAttempts = Math.max(0, parseInt(retry, 10)) + 1;
18  178
19  179 for (let attempt = 1; attempt <= totalAttempts; attempt += 1) {
20  180     try {
21  181         console.log(`[cloudCall][${pageTag}] calling "${name}" (${attempt}/${totalAttempts})
22      ↳ `, payload);
23  182         const result = await invokeCloudFunction(method, {
24  183             context,
25  184             options: {
26  185                 name,
27  186                 data: payload
28  187             },
29  188             customReqOpts: typeof timeoutMs === 'number' && timeoutMs > 0 ? { timeout: timeo
30      ↳ utMs } : undefined
31  189         });
32  190         console.log(`[cloudCall][${pageTag}] "${name}" succeeded`, result);
33  191         return result;
34  192     } catch (error) {
35  193         const isLastAttempt = attempt === totalAttempts;
36  194         const errorCode = error && error.code ? error.code : error?.errCode;
37  195
38  196         console.error(`[cloudCall][${pageTag}] "${name}" failed (${attempt}/${totalAttempts}
39      ↳ ): ${formatErrorForLog(error)}`);
40  197
41  198         if (errorCode === 'NO_OPENID') {
42  199             throw createError('NO_OPENID', '\u7528\u6237\u672A\u767B\u5F55\u6216 openid \u7F
43      ↳ 3A\u5931', error);
44  200         }
45  201
46  202         if (isLastAttempt) {
47  203             const finalError = createError(errorCode || 'CLOUD_CALL_FAILED', '\u4E91\u51FD\u
48      ↳ 6570\u8C03\u7528\u5931\u8D25', error);
49  204             if (typeof uni !== 'undefined' && uni.showToast) {
50  205                 uni.showToast({
```

01	206	title: '\u7F51\u7EDC\u5F02\u5E38\uFF0C\u8BF7\u7A0D\u540E\u518D\u8BD5',
02	207	icon: 'none'
03	208	});
04	209	}
05	210	throw finalError;
06	211	}
07	212	
08	213	if (retryDelay > 0) {
09	214	await new Promise((resolve) => setTimeout(resolve, retryDelay));
10	215	}
11	216	}
12	217	}
13	218	
14	219	throw createError('CLOUD_CALL_FAILED', '\u4E91\u51FD\u6570\u8C03\u7528\u5931\u8D25');
15	220	}
16	221	
17	222	export default {
18	223	cloudCall
19	224	};
20	**FILE**	**FILE 04 | api-cache/_shared/cloud-wrapper.js |** 语义 **API** 解包与错误归一
21	1	import { cloudCall } from '../../utils/cloudCall.js';
22	2	
23	3	function getResult(res) {
24	4	if (res && typeof res === 'object') {
25	5	if (res.result && typeof res.result === 'object') {
26	6	return res.result;
27	7	}
28	8	return res;
29	9	}
30	10	return {};
31	11	}
32	12	
33	13	function isSuccessResult(result) {
34	14	return !!result && (result.success === true || result.code === 0);
35	15	}
36	16	
37	17	function extractErrorMessage(value) {
38	18	if (!value) {
39	19	return '';
40	20	}
41	21	if (value instanceof Error) {
42	22	return value.message || String(value);
43	23	}
44	24	if (typeof value === 'string') {
45	25	return value;
46	26	}
47	27	if (typeof value === 'object') {
48	28	if (typeof value.message === 'string' && value.message) {
49	29	return value.message;
50	30	}

```
01  31    if (typeof value.msg === 'string' && value.msg) {
02  32      return value.msg;
03  33    }
04  34    if (typeof value.errMsg === 'string' && value.errMsg) {
05  35      return value.errMsg;
06  36    }
07  37    try {
08  38      return JSON.stringify(value);
09  39    } catch (_) {
10  40      return String(value);
11  41    }
12  42  }
13  43  return String(value);
14  44}
15  45
16  46function unwrapResult(res, fallbackMessage = '操作失败') {
17  47  const result = getResult(res);
18  48  if (!isSuccessResult(result)) {
19  49    const message =
20  50      extractErrorMessage(result.error) ||
21  51      extractErrorMessage(result.message) ||
22  52      extractErrorMessage(result.msg) ||
23  53      extractErrorMessage(result.errMsg) ||
24  54      fallbackMessage;
25  55    const error = new Error(message);
26  56    error.result = result;
27  57    throw error;
28  58  }
29  59  return result;
30  60}
31  61
32  62async function callCloudAndUnwrap(name, payload = {}, options = {}, fallbackMessage) {
33  63  const res = await cloudCall(name, payload, options);
34  64  return unwrapResult(res, fallbackMessage);
35  65}
36  66
37  67async function callCloudAndGetResult(name, payload = {}, options = {}) {
38  68  const res = await cloudCall(name, payload, options);
39  69  return getResult(res);
40  70}
41  71
42  72async function callActionAndUnwrap({
43  73  functionName,
44  74  action,
45  75  payload = {},
46  76  pageTag,
47  77  context,
48  78  requireAuth = true,
49  79  fallbackMessage = '操作失败'
50  80} = {}) {
```

```
01  81    return callCloudAndUnwrap(
02  82      functionName,
03  83      {
04  84        action,
05  85        ...payload
06  86      },
07  87      {
08  88        pageTag: pageTag || `${functionName}:${action}`,
09  89        context,
10  90        requireAuth
11  91      },
12  92      fallbackMessage
13  93    );
14  94  }
15  95
16  96  function createActionCaller({
17  97    functionName,
18  98    pageTagPrefix,
19  99    requireAuth = true,
20 100    defaultFallbackMessage = '操作失败'
21 101  } = {}) {
22 102    return async function callAction(action, payload = {}, options = {}) {
23 103      const {
24 104        pageTag,
25 105        context,
26 106        fallbackMessage = defaultFallbackMessage,
27 107        requireAuth: required = requireAuth
28 108      } = options;
29 109      return callActionAndUnwrap({
30 110        functionName,
31 111        action,
32 112        payload,
33 113        pageTag: pageTag || `${pageTagPrefix || functionName}:${action}`,
34 114        context,
35 115        requireAuth: required,
36 116        fallbackMessage
37 117      });
38 118    };
39 119  }
40 120
41 121  const cloudWrapper = {
42 122    getResult,
43 123    isSuccessResult,
44 124    unwrapResult,
45 125    callCloudAndGetResult,
46 126    callCloudAndUnwrap,
47 127    callActionAndUnwrap,
48 128    createActionCaller
49 129  };
50 130
```

01	131	export {
02	132	getResult,
03	133	isSuccessResult,
04	134	unwrapResult,
05	135	callCloudAndGetResult,
06	136	callCloudAndUnwrap,
07	137	callActionAndUnwrap,
08	138	createActionCaller
09	139	};
10	140	
11	141	export default cloudWrapper;
12	142	
13	143	if (typeof module !== 'undefined' && module.exports) {
14	144	module.exports = cloudWrapper;
15	145	module.exports.default = cloudWrapper;
16	146	}
17	**FILE**	**FILE 05 | functions/login/index.js |** 登录云函数入口
18	1	// 云函数入口文件 - 兼容微信云开发和CloudBase
19	2	const cloud = require('wx-server-sdk');
20	3	
21	4	cloud.init({
22	5	env: cloud.DYNAMIC_CURRENT_ENV
23	6	});
24	7	
25	8	/**
26	9	* 通用login函数，处理来自小程序、App、Web的调用
27	10	*/
28	11	exports.main = async (event, context) => {
29	12	try {
30	13	console.log('🔍 [云函数] login被调用，event:', event);
31	14	console.log('🔍 [云函数] login被调用，context:', context);
32	15	
33	16	const wxContext = cloud.getWXContext();
34	17	console.log('🔍 [云函数] wxContext:', wxContext);
35	18	
36	19	// 在H5环境下，wxContext.OPENID可能为空，使用context中的信息
37	20	const openid = wxContext.OPENID || context.OPENID || 'anonymous_' + Date.now();
38	21	const appid = wxContext.APPID || context.APPID || 'unknown';
39	22	const unionid = wxContext.UNIONID || context.UNIONID || null;
40	23	
41	24	console.log('🔍 [云函数] 最终openid:', openid);
42	25	
43	26	// 返回兼容格式，同时支持微信云开发和CloudBase
44	27	const result = {
45	28	// 微信云开发格式（小程序兼容）
46	29	event,
47	30	openid: openid,
48	31	appid: appid,
49	32	unionid: unionid,
50	33	

```
01  34      // CloudBase格式（H5兼容）
02  35      success: true,
03  36      uid: openid, // 使用openid作为uid
04  37      result: {
05  38        openid: openid,
06  39        uid: openid,
07  40        appid: appid,
08  41        unionid: unionid
09  42      }
10  43    };
11  44
12  45    console.log('🔍 [云函数] 返回结果:', result);
13  46    return result;
14  47  } catch (error) {
15  48    console.error('登录云函数错误:', error);
16  49    return {
17  50      success: false,
18  51      error: error.message,
19  52      result: null
20  53    };
21  54  }
22  55 };
23  FILE FILE 06 | functions/getUserProfile/index.js | 用户资料查询云函数
24  1  // 云函数入口文件
25  2  const cloud = require('wx-server-sdk');
26  3  const { ensureUserDefaultAvatar } = require('../_lib/default-avatar');
27  4
28  5  cloud.init({
29  6    env: cloud.DYNAMIC_CURRENT_ENV
30  7  });
31  8
32  9  const db = cloud.database();
33  10 const $ = db.command.aggregate;
34  11
35  12 function normalizeAppBackgroundUrl(url) {
36  13   return typeof url === 'string' ? url.trim() : '';
37  14 }
38  15
39  16 function normalizeAppBackgroundMode(mode, backgroundUrl = '') {
40  17   const normalizedUrl = normalizeAppBackgroundUrl(backgroundUrl);
41  18   if (!normalizedUrl) {
42  19     return '';
43  20   }
44  21   return typeof mode === 'string' && mode.trim().toLowerCase() === 'header'
45  22     ? 'header'
46  23     : 'full';
47  24 }
48  25
49  26 // 云函数入口函数
50  27 exports.main = async (event, context) => {
```

```
01  28  const wxContext = cloud.getWXContext();
02  29  const currentOpenid = wxContext.OPENID || event.openid;
03  30
04  31  if (!currentOpenid) {
05  32
06  33    return {
07  34      success: false,
08  35      message: '无法获取用户 openid，请重新登录',
09  36      code: 'NO_OPENID'
10  37    };
11  38  }
12  39
13  40  const { userId, skip = 0, limit = 20, onlyProfile = false } = event;
14  41
15  42  if (!userId) {
16  43    return { success: false, message: '用户ID不能为空' };
17  44  }
18  45
19  46  try {
20  47    // 检查是否屏蔽了目标用户
21  48    let isBlocked = false;
22  49    try {
23  50      const blockRes = await db.collection('blocks').where({
24  51        blockerId: currentOpenid,
25  52        blockedId: userId
26  53      }).limit(1).get();
27  54      isBlocked = blockRes.data.length > 0;
28  55
29  56      // 如果当前用户屏蔽了目标用户，返回错误
30  57      if (isBlocked) {
31  58        return {
32  59          success: false,
33  60          message: '无法查看该用户的信息',
34  61          code: 'USER_BLOCKED'
35  62        };
36  63      }
37  64    } catch (blockError) {
38  65      console.error('检查屏蔽状态失败:', blockError);
39  66    }
40  67
41  68    // 获取被屏蔽的用户ID列表（用于过滤帖子，使用缓存）
42  69    let blockedUserIds = [];
43  70    try {
44  71      const getBlockedUserIds = require('./_lib/get-blocked-user-ids');
45  72      blockedUserIds = await getBlockedUserIds(currentOpenid, db);
46  73    } catch (blockError) {
47  74      console.error('获取屏蔽列表失败:', blockError);
48  75    }
49  76
50  77    // 获取目标用户的公开信息（如果只需要用户信息，跳过帖子查询）
```

```
01  78    let profileData;
02  79    if (onlyProfile) {
03  80      // 轻量级模式：只获取用户信息，不查询帖子
04  81      profileData = await db.collection('users')
05  82        .where({ _openid: userId })
06  83        .field({
07  84          _id: 1,
08  85          _openid: 1,
09  86          nickName: 1,
10  87          avatarUrl: 1,
11  88          bio: 1,
12  89          occupation: 1,
13  90          region: 1,
14  91          signatureUrl: 1,
15  92          appBackgroundUrl: 1,
16  93          appBackgroundMode: 1,
17  94          poemId: 1,
18  95          growthCounts: 1
19  96        })
20  97        .limit(1)
21  98        .get();
22  99
23 100      // 转换为与 aggregate 相同的格式
24 101      profileData = { list: profileData.data };
25 102    } else {
26 103      // 完整模式：获取用户信息和帖子
27 104      profileData = await db.collection('users').aggregate()
28 105        .match({ _openid: userId })
29 106        .limit(1)
30 107        .lookup({
31 108          from: 'posts',
32 109          let: { user_openid: '$_openid' },
33 110          pipeline: [
34 111            {
35 112              $match: {
36 113                $expr: {
37 114                  $and: [
38 115                    { $eq: ['$_openid', '$$user_openid'] },
39 116                    { $ne: ['$isActivityPost', true] }
40 117                  ]
41 118                }
42 119              }
43 120            },
44 121            { $sort: { createTime: -1 } },
45 122            { $skip: skip },
46 123            { $limit: limit }
47 124          ],
48 125          as: 'posts'
49 126        })
50 127        .project({
```

```
01  128         _id: 1,
02  129         _openid: 1,
03  130         nickName: 1,
04  131         avatarUrl: 1,
05  132         bio: 1,
06  133         occupation: 1,
07  134         region: 1,
08  135         signatureUrl: 1,
09  136         appBackgroundUrl: 1,
10  137         appBackgroundMode: 1,
11  138         poemId: 1,
12  139         growthCounts: 1,
13  140         // 不返回隐私信息（生日、年龄等）
14  141         posts: 1
15  142       })
16  143       .end();
17  144   }
18  145
19  146   if (!profileData.list || profileData.list.length === 0) {
20  147     return { success: false, message: '用户不存在' };
21  148   }
22  149
23  150   const userInfo = profileData.list[0];
24  151   userInfo.avatarUrl = await ensureUserDefaultAvatar({ db, openid: userId, user: userInfo });
25  152   const canViewAppBackground = String(currentOpenid) === String(userId);
26  153   if (!canViewAppBackground) {
27  154     userInfo.appBackgroundUrl = '';
28  155     userInfo.appBackgroundMode = '';
29  156   }
30  157   let posts = onlyProfile ? [] : (userInfo.posts || []);
31  158
32  159   // 只有在非轻量级模式下才处理帖子
33  160   if (!onlyProfile) {
34  161     // 非本人访问时，过滤掉隐藏帖和被屏蔽用户的帖子
35  162     try {
36  163       const isOwner = String(currentOpenid) === String(userId);
37  164       if (!isOwner && Array.isArray(posts)) {
38  165         posts = posts.filter((p) => {
39  166           if (!p || p.isHidden === true) return false;
40  167           if (p.isActivityPost === true) return false;
41  168           // 虽然已经检查过是否屏蔽了目标用户，但这里再过滤一次确保安全
42  169           if (blockedUserIds.length > 0 && blockedUserIds.includes(p._openid)) return false;
43  170           return true;
44  171         });
45  172       }
46  173     } catch (filterError) {
47  174       console.error('过滤帖子失败:', filterError);
48  175     }
49  176
50  177     // 处理图片URL
```

```
01  178     posts.forEach(post => {
02  179       if (!Array.isArray(post.imageUrls)) {
03  180         post.imageUrls = post.imageUrls ? [post.imageUrls] : [];
04  181       }
05  182       if (!Array.isArray(post.originalImageUrls)) {
06  183         post.originalImageUrls = post.originalImageUrls ? [post.originalImageUrls] : [];
07  184       }
08  185     });
09  186
10  187     // 转换云存储URL为临时URL
11  188     const fileIDs = [];
12  189     posts.forEach(post => {
13  190       if (post.imageUrl && post.imageUrl.startsWith('cloud://')) {
14  191         fileIDs.push(post.imageUrl);
15  192       }
16  193       if (post.imageUrls && Array.isArray(post.imageUrls)) {
17  194         post.imageUrls.forEach(url => {
18  195           if (url && url.startsWith('cloud://')) {
19  196             fileIDs.push(url);
20  197           }
21  198         });
22  199       }
23  200       if (post.originalImageUrls && Array.isArray(post.originalImageUrls)) {
24  201         post.originalImageUrls.forEach(url => {
25  202           if (url && url.startsWith('cloud://')) {
26  203             fileIDs.push(url);
27  204           }
28  205         });
29  206       }
30  207     });
31  208
32  209     // 处理用户头像URL和签名URL
33  210     if (userInfo.avatarUrl && userInfo.avatarUrl.startsWith('cloud://')) {
34  211       fileIDs.push(userInfo.avatarUrl);
35  212     }
36  213     if (userInfo.signatureUrl && userInfo.signatureUrl.startsWith('cloud://')) {
37  214       fileIDs.push(userInfo.signatureUrl);
38  215     }
39  216     if (userInfo.appBackgroundUrl && userInfo.appBackgroundUrl.startsWith('cloud://')) {
40  217       fileIDs.push(userInfo.appBackgroundUrl);
41  218     }
42  219
43  220     if (fileIDs.length > 0) {
44  221       try {
45  222         const fileListResult = await cloud.getTempFileURL({ fileList: fileIDs });
46  223         const urlMap = new Map();
47  224         fileListResult.fileList.forEach(item => {
48  225           if (item.status === 0) {
49  226             urlMap.set(item.fileID, item.tempFileURL);
50  227           }
```

```
01  228          });
02  229
03  230          // 转换帖子图片URL
04  231          posts.forEach(post => {
05  232            if (post.imageUrl && urlMap.has(post.imageUrl)) {
06  233              post.imageUrl = urlMap.get(post.imageUrl);
07  234            }
08  235            if (post.imageUrls && Array.isArray(post.imageUrls)) {
09  236              post.imageUrls = post.imageUrls.map(url => {
10  237                return urlMap.has(url) ? urlMap.get(url) : url;
11  238              });
12  239            }
13  240            if (post.originalImageUrls && Array.isArray(post.originalImageUrls)) {
14  241              post.originalImageUrls = post.originalImageUrls.map(url => {
15  242                return urlMap.has(url) ? urlMap.get(url) : url;
16  243              });
17  244            }
18  245          });
19  246
20  247          // 转换用户头像URL和签名URL
21  248          if (userInfo.avatarUrl && urlMap.has(userInfo.avatarUrl)) {
22  249            userInfo.avatarUrl = urlMap.get(userInfo.avatarUrl);
23  250          } else if (userInfo.avatarUrl && userInfo.avatarUrl.startsWith('cloud://')) {
24  251            userInfo.avatarUrl = '/static/images/avatar.png';
25  252          }
26  253          if (userInfo.signatureUrl && urlMap.has(userInfo.signatureUrl)) {
27  254            userInfo.signatureUrl = urlMap.get(userInfo.signatureUrl);
28  255          } else if (userInfo.signatureUrl && userInfo.signatureUrl.startsWith('cloud://')) {
29  256            userInfo.signatureUrl = null;
30  257          }
31  258          if (userInfo.appBackgroundUrl && urlMap.has(userInfo.appBackgroundUrl)) {
32  259            userInfo.appBackgroundUrl = urlMap.get(userInfo.appBackgroundUrl);
33  260          } else if (userInfo.appBackgroundUrl && userInfo.appBackgroundUrl.startsWith('cloud://
34    ↳ ')) {
35  261            userInfo.appBackgroundUrl = '';
36  262          }
37  263        } catch (fileError) {
38  264          console.error('文件URL转换失败:', fileError);
39  265        }
40  266      }
41  267
42  268      // 批量获取所有帖子的评论数量（优化：并行查询而不是串行）
43  269      if (posts.length > 0) {
44  270        const postIds = posts.map(p => p._id);
45  271        try {
46  272          // 使用 aggregate 批量统计评论数
47  273          const commentCountsAgg = await db.collection('comments').aggregate()
48  274            .match({
49  275              postId: db.command.in(postIds)
50  276            })
```

```
01  277           .group({
02  278             _id: '$postId',
03  279             count: $.sum(1)
04  280           })
05  281           .end();
06  282
07  283         // 创建评论数映射
08  284         const commentCountMap = new Map();
09  285         if (commentCountsAgg.list && commentCountsAgg.list.length > 0) {
10  286           commentCountsAgg.list.forEach(item => {
11  287             commentCountMap.set(item._id, item.count);
12  288           });
13  289         }
14  290
15  291         // 为每个帖子设置评论数
16  292         posts.forEach(post => {
17  293           post.commentCount = commentCountMap.get(post._id) || 0;
18  294         });
19  295       } catch (err) {
20  296         console.error('批量获取评论数量失败:', err);
21  297         // 失败时设置默认值
22  298         posts.forEach(post => {
23  299           post.commentCount = 0;
24  300         });
25  301       }
26  302
27  303       // 检查当前用户是否点赞了这些帖子
28  304       try {
29  305         const votesResult = await db.collection('votes_log')
30  306           .where({
31  307             _openid: currentOpenid,
32  308             postId: db.command.in(postIds)
33  309           })
34  310           .get();
35  311
36  312         const votedPostIds = new Set(votesResult.data.map(v => v.postId));
37  313         posts.forEach(post => {
38  314           post.isVoted = votedPostIds.has(post._id);
39  315         });
40  316       } catch (err) {
41  317         console.error('获取点赞状态失败:', err);
42  318         posts.forEach(post => {
43  319           post.isVoted = false;
44  320         });
45  321       }
46  322     }
47  323   }
48  324
49  325   // 处理用户头像和签名URL（轻量级模式或posts为空时）
50  326   if (onlyProfile || posts.length === 0) {
```

```
01  327     const fileIDs = [];
02  328     if (userInfo.avatarUrl && userInfo.avatarUrl.startsWith('cloud://')) {
03  329       fileIDs.push(userInfo.avatarUrl);
04  330     }
05  331     if (userInfo.signatureUrl && userInfo.signatureUrl.startsWith('cloud://')) {
06  332       fileIDs.push(userInfo.signatureUrl);
07  333     }
08  334     if (userInfo.appBackgroundUrl && userInfo.appBackgroundUrl.startsWith('cloud://')) {
09  335       fileIDs.push(userInfo.appBackgroundUrl);
10  336     }
11  337
12  338     if (fileIDs.length > 0) {
13  339       try {
14  340         const fileListResult = await cloud.getTempFileURL({ fileList: fileIDs });
15  341         const urlMap = new Map();
16  342         fileListResult.fileList.forEach(item => {
17  343           if (item.status === 0) {
18  344             urlMap.set(item.fileID, item.tempFileURL);
19  345           }
20  346         });
21  347
22  348         if (userInfo.avatarUrl && urlMap.has(userInfo.avatarUrl)) {
23  349           userInfo.avatarUrl = urlMap.get(userInfo.avatarUrl);
24  350         }
25  351         if (userInfo.signatureUrl && urlMap.has(userInfo.signatureUrl)) {
26  352           userInfo.signatureUrl = urlMap.get(userInfo.signatureUrl);
27  353         }
28  354         if (userInfo.appBackgroundUrl && urlMap.has(userInfo.appBackgroundUrl)) {
29  355           userInfo.appBackgroundUrl = urlMap.get(userInfo.appBackgroundUrl);
30  356         }
31  357       } catch (fileError) {
32  358         console.error('文件URL转换失败:', fileError);
33  359       }
34  360     }
35  361   }
36  362
37  363   userInfo.appBackgroundMode = normalizeAppBackgroundMode(userInfo.appBackgroundMode, userInfo
38    ↳ .appBackgroundUrl);
39  364
40  365   return {
41  366     success: true,
42  367           userInfo: {
43  368       _openid: userInfo._openid,
44  369       nickName: userInfo.nickName || '微信用户',
45  370       avatarUrl: userInfo.avatarUrl || '',
46  371       occupation: userInfo.occupation || '',
47  372       region: userInfo.region || '',
48  373       bio: userInfo.bio || '这个人很懒，什么都没有写...',
49  374       signatureUrl: userInfo.signatureUrl || '',
50  375       appBackgroundUrl: userInfo.appBackgroundUrl || '',
```

```
01  724         }
02  725       });
03  726
04  727       // 兼容写入：高光行数组
05  728       try {
06  729         await db.collection('posts').doc(result._id).update({
07  730           data: {
08  731             highlightLines: Array.isArray(effectiveHighlightLines) ? effectiveHighlightLines : [
09    ↳ ]
10  732           }
11  733         });
12  734       } catch (e) {
13  735         console.warn('[contentCheck] 写入 highlightLines 失败（忽略）:', e);
14  736       }
15  737     } catch (e) {
16  738       console.warn('[contentCheck] 追加写入颜色/高光句失败，不影响发布:', e);
17  739     }
18  740
19  741     console.log('数据库写入成功，返回结果:', {
20  742       postId: result._id,
21  743       insertedCount: result.stats?.inserted || 1
22  744     });
23  745
24  746     // ------------------- 4. 非原创诗歌自动创建诗人主页 -------------------
25  747     if ((publishMode === 'poem' || isSeries) && !isOriginal && authorName && authorName.trim())
26    ↳ {
27  748       try {
28  749         const poetName = authorName.trim();
29  750         console.log('检查诗人主页是否存在:', poetName);
30  751
31  752         // 查询诗人是否已存在
32  753         const poetResult = await db.collection('poets')
33  754           .where({ name: poetName })
34  755           .limit(1)
35  756           .get();
36  757
37  758         if (!poetResult.data || poetResult.data.length === 0) {
38  759           // 诗人不存在，自动创建
39  760           console.log('诗人主页不存在，自动创建:', poetName);
40  761           const newPoet = {
41  762             name: poetName,
42  763             avatar: '',  // 默认无头像
43  764             bio: '',     // 默认无简介
44  765             createTime: db.serverDate(),
45  766             updateTime: db.serverDate(),
46  767             creatorOpenid: currentOpenid  // 记录谁首次触发创建
47  768           };
48  769
49  770           const poetAddResult = await db.collection('poets').add({ data: newPoet });
50  771           console.log('诗人主页创建成功:', poetAddResult._id);
```

01	772	} else {
02	773	console.log('诗人主页已存在，无需创建:', poetName);
03	774	}
04	775	} catch (poetError) {
05	776	// 诗人主页创建失败不影响帖子发布
06	777	console.warn('自动创建诗人主页失败（不影响发帖）:', poetError);
07	778	}
08	779	}
09	780	
10	781	// 全部成功，返回成功状态
11	782	return buildSuccess({
12	783	postId: result._id,
13	784	msg: '发布成功'
14	785	});
15	786	
16	787	} catch (dbError) {
17	788	console.error('数据库写入失败:', dbError);
18	789	console.error('数据库错误详情:', {
19	790	message: dbError.message,
20	791	code: dbError.code,
21	792	stack: dbError.stack
22	793	});
23	794	return buildFailure({
24	795	code: -3,
25	796	msg: '数据库存储失败: ' + dbError.message,
26	797	errorCode: dbError.code || 'DB_WRITE_FAILED',
27	798	extra: { error: dbError.message }
28	799	});
29	800	}
30	801	};
31	FILE	**FILE 12 | functions/createPost/index.js |** 创建帖子云函数
32	1	// 云函数：createPost
33	2	// 功能：创建新帖子
34	3	
35	4	const cloud = require('wx-server-sdk');
36	5	
37	6	cloud.init({
38	7	env: cloud.DYNAMIC_CURRENT_ENV
39	8	});
40	9	
41	10	const db = cloud.database();
42	11	
43	12	exports.main = async (event, context) => {
44	13	console.log('🔍 [createPost] 开始创建帖子');
45	14	console.log('🔍 [createPost] 请求参数:', JSON.stringify(event, null, 2));
46	15	
47	16	const wxContext = cloud.getWXContext();
48	17	const openid = event.openid || wxContext.OPENID;
49	18	
50	19	if (!openid) {

```
01  20    return {
02  21      code: -1,
03  22      msg: '无法获取用户信息，请重新登录'
04  23    };
05  24  }
06  25
07  26  try {
08  27    // 获取用户信息
09  28    const userResult = await db.collection('users').where({
10  29      _openid: openid
11  30    }).get();
12  31
13  32    if (userResult.data.length === 0) {
14  33      return {
15  34        code: -1,
16  35        msg: '用户信息不存在'
17  36      };
18  37    }
19  38
20  39    const user = userResult.data[0];
21  40
22  41    // 准备帖子数据
23  42    const postData = {
24  43      _openid: openid,
25  44      authorName: event.isAnonymous ? event.anonymousAuthorName : user.nickName,
26  45      authorAvatar: event.isAnonymous ? '/static/images/avatar.png' : user.avatarUrl,
27  46      title: event.title || '',
28  47      content: event.content || '',
29  48      createTime: db.serverDate(),
30  49      votes: 0,
31  50      commentCount: 0,
32  51      viewCount: 0,
33  52      isPoem: event.publishMode === 'poem' || event.isSeries || false,
34  53      isSeries: event.isSeries || false,
35  54      isOriginal: event.isOriginal || false,
36  55      isDiscussion: event.isDiscussion || false,
37  56      author: event.author || '',
38  57      tags: event.tags || [],
39  58      isAnonymous: event.isAnonymous || false,
40  59      anonymousAuthorName: event.anonymousAuthorName || '匿名用户',
41  60      realAuthorOpenid: event.realAuthorOpenid || null
42  61    };
43  62
44  63    // 添加颜色信息
45  64    if (event.backgroundColor) {
46  65      postData.backgroundColor = event.backgroundColor;
47  66    }
48  67    if (event.textColor) {
49  68      postData.textColor = event.textColor;
50  69    }
```

```
01  70
02  71    // 添加高光行信息
03  72    if (event.highlightLines && event.highlightLines.length > 0) {
04  73      postData.highlightLines = event.highlightLines;
05  74      postData.highlightSentence = event.highlightLines[0];
06  75    }
07  76
08  77    // 添加图片信息
09  78    if (event.fileIDs && event.fileIDs.length > 0) {
10  79      postData.imageUrl = event.fileIDs[0];
11  80      postData.imageUrls = event.fileIDs;
12  81    }
13  82    if (event.originalFileIDs && event.originalFileIDs.length > 0) {
14  83      postData.originalImageUrl = event.originalFileIDs[0];
15  84      postData.originalImageUrls = event.originalFileIDs;
16  85    }
17  86
18  87    // 讨论模式特殊处理
19  88    if (event.isDiscussion) {
20  89      postData.sentenceGroups = event.sentenceGroups || [];
21  90      postData.discussionSentences = event.discussionSentences || [];
22  91    }
23  92
24  93    // 组诗模式特殊处理
25  94    if (event.isSeries) {
26  95      postData.seriesBlocks = event.seriesBlocks || [];
27  96      postData.seriesBlockCount = (event.seriesBlocks || []).length;
28  97    }
29  98
30  99    // 创建帖子
31 100    const result = await db.collection('posts').add({
32 101      data: postData
33 102    });
34 103
35 104    console.log('✅ [createPost] 帖子创建成功:', result._id);
36 105
37 106    return {
38 107      code: 0,
39 108      msg: '发布成功',
40 109      postId: result._id
41 110    };
42 111
43 112  } catch (error) {
44 113    console.error('❌ [createPost] 创建帖子失败:', error);
45 114    return {
46 115      code: -1,
47 116      msg: error.message || '发布失败'
48 117    };
49 118  }
50 119};
```

01	FILE	FILE 13 | functions/getPostList/index.js | 帖子列表云函数
02	1	// 云函数入口文件
03	2	const cloud = require('wx-server-sdk');
04	3	
05	4	cloud.init({
06	5	env: cloud.DYNAMIC_CURRENT_ENV
07	6	});
08	7	
09	8	const db = cloud.database();
10	9	const _ = db.command;
11	10	const $ = _.aggregate;
12	11	
13	12	// 频率限制配置
14	13	const RATE_LIMITS = {
15	14	perMinute: 20, // 每分钟20次
16	15	perHour: 200, // 每小时200次
17	16	perDay: 600 // 每天600次
18	17	};
19	18	
20	19	// 频率限制检查函数
21	20	async function checkRateLimit(openid) {
22	21	try {
23	22	const now = Date.now();
24	23	const minuteAgo = now - 60 * 1000;
25	24	const hourAgo = now - 60 * 60 * 1000;
26	25	const dayAgo = now - 24 * 60 * 60 * 1000;
27	26	
28	27	console.log('🔍 [RateLimit] 开始检查频率限制 - openid:', openid.substring(0, 8) + '...');
29	28	
30	29	// 检查分钟级限制
31	30	const minuteCount = await db.collection('api_rate_limits')
32	31	.where({
33	32	openid: openid,
34	33	api_name: 'getPostList',
35	34	timestamp: _.gte(minuteAgo)
36	35	})
37	36	.count();
38	37	
39	38	if (minuteCount.total >= RATE_LIMITS.perMinute) {
40	39	console.log('🚫 [RateLimit] 分钟级限制超限:', minuteCount.total, '/', RATE_LIMITS.perMinut
41	↳	e);
42	40	return {
43	41	allowed: false,
44	42	reason: 'RATE_LIMIT_MINUTE_EXCEEDED',
45	43	resetTime: minuteAgo + 60000,
46	44	limit: RATE_LIMITS.perMinute,
47	45	current: minuteCount.total,
48	46	waitSeconds: Math.ceil((minuteAgo + 60000 - now) / 1000)
49	47	};
50	48	}

```
01  49
02  50    // 检查小时级限制
03  51    const hourCount = await db.collection('api_rate_limits')
04  52      .where({
05  53        openid: openid,
06  54        api_name: 'getPostList',
07  55        timestamp: _.gte(hourAgo)
08  56      })
09  57      .count();
10  58
11  59    if (hourCount.total >= RATE_LIMITS.perHour) {
12  60      console.log('🚫 [RateLimit] 小时级限制超限:', hourCount.total, '/', RATE_LIMITS.perHour);
13  61      return {
14  62        allowed: false,
15  63        reason: 'RATE_LIMIT_HOUR_EXCEEDED',
16  64        resetTime: hourAgo + 3600000,
17  65        limit: RATE_LIMITS.perHour,
18  66        current: hourCount.total,
19  67        waitSeconds: Math.ceil((hourAgo + 3600000 - now) / 1000)
20  68      };
21  69    }
22  70
23  71    // 检查日级限制
24  72    const dayCount = await db.collection('api_rate_limits')
25  73      .where({
26  74        openid: openid,
27  75        api_name: 'getPostList',
28  76        timestamp: _.gte(dayAgo)
29  77      })
30  78      .count();
31  79
32  80    if (dayCount.total >= RATE_LIMITS.perDay) {
33  81      console.log('🚫 [RateLimit] 日级限制超限:', dayCount.total, '/', RATE_LIMITS.perDay);
34  82      return {
35  83        allowed: false,
36  84        reason: 'RATE_LIMIT_DAY_EXCEEDED',
37  85        resetTime: dayAgo + 86400000,
38  86        limit: RATE_LIMITS.perDay,
39  87        current: dayCount.total,
40  88        waitSeconds: Math.ceil((dayAgo + 86400000 - now) / 1000)
41  89      };
42  90    }
43  91
44  92    // 记录本次请求
45  93    await db.collection('api_rate_limits').add({
46  94      data: {
47  95        openid: openid,
48  96        api_name: 'getPostList',
49  97        timestamp: now
50  98      }
```

```
01  99     });
02  100
03  101    // 定期清理过期记录（1%概率执行）
04  102    if (Math.random() < 0.01) {
05  103      try {
06  104        await db.collection('api_rate_limits')
07  105          .where({
08  106            timestamp: _.lt(dayAgo)
09  107          })
10  108          .remove();
11  109        console.log('🧹 [RateLimit] 清理过期记录完成');
12  110      } catch (cleanupError) {
13  111        console.warn('⚠️ [RateLimit] 清理过期记录失败:', cleanupError);
14  112      }
15  113    }
16  114
17  115    console.log('✅ [RateLimit] 频率检查通过 - 当前分钟:', minuteCount.total + 1, '小时:', hourCo
18  ↳  unt.total + 1, '日:', dayCount.total + 1);
19  116    return {
20  117      allowed: true,
21  118      currentCounts: {
22  119        perMinute: minuteCount.total + 1,
23  120        perHour: hourCount.total + 1,
24  121        perDay: dayCount.total + 1
25  122      }
26  123    };
27  124
28  125  } catch (error) {
29  126    console.error('❌ [RateLimit] 频率限制检查失败:', error);
30  127    // 检查失败时允许通过，避免影响正常用户
31  128    return { allowed: true };
32  129  }
33  130}
34  131
35  132// 云函数入口函数
36  133exports.main = async (event, context) => {
37  134  console.log('🔍 [getPostList] ========== 云函数开始执行 ==========');
38  135  console.log('🔍 [getPostList] 接收到的参数:', JSON.stringify(event, null, 2));
39  136
40  137  const wxContext = cloud.getWXContext();
41  138  const wxCtxOpenid = wxContext.OPENID;
42  139  const eventOpenid = event.openid;
43  140  const openid = eventOpenid || wxCtxOpenid;
44  141  // 添加 author 参数用于诗人筛选
45  142  const { skip = 0, limit = 10, isPoem, isOriginal, isDiscussion, tag = '', author = '', include
46  ↳  Activity = false, activityId = '' } = event;
47  143  const safeActivityId = typeof activityId === 'string' ? activityId.trim() : '';
48  144  const isActivityQuery = includeActivity === true && !!safeActivityId;
49  145
50  146  console.log('🔍 [getPostList] 解析参数:', {
```

```
01  147     eventOpenid: eventOpenid ? 'provided' : 'missing',
02  148     wxCtxOpenid: wxCtxOpenid ? 'provided' : 'missing',
03  149     chosenOpenidSource: eventOpenid ? 'event.openid' : 'wxContext.OPENID',
04  150     chosenOpenidExists: !!openid,
05  151     skip,
06  152     limit,
07  153     isPoem,
08  154     isOriginal,
09  155     isDiscussion,
10  156     tag,
11  157     author,
12  158     includeActivity,
13  159     activityId: safeActivityId
14  160   });
15  161
16  162   if (!openid) {
17  163     console.error('❌ [getPostList] 无法获取用户 openid');
18  164     return {
19  165       success: false,
20  166       message: '无法获取用户 openid，请重新登录',
21  167       code: 'NO_OPENID'
22  168     };
23  169   }
24  170
25  171   if (includeActivity === true && !safeActivityId) {
26  172     return {
27  173       success: false,
28  174       message: 'activityId 不能为空',
29  175       code: 'INVALID_ACTIVITY_ID'
30  176     };
31  177   }
32  178
33  179   // 检查请求频率限制（临时禁用以排查超时问题）
34  180   // TODO: 优化频率限制检查逻辑后重新启用
35  181   /*
36  182   const rateLimitResult = await checkRateLimit(openid);
37  183   if (!rateLimitResult.allowed) {
38  184     console.log('🚫 [getPostList] 频率限制拦截:', rateLimitResult.reason);
39  185     return {
40  186       success: false,
41  187       code: rateLimitResult.reason,
42  188       message: `请求过于频繁，请在${rateLimitResult.waitSeconds}秒后重试`,
43  189       data: {
44  190         limit: rateLimitResult.limit,
45  191         current: rateLimitResult.current,
46  192         resetTime: rateLimitResult.resetTime,
47  193         waitSeconds: rateLimitResult.waitSeconds
48  194       }
49  195     };
50  196   }
```

```
01  197  */
02  198
03  199  try {
04  200    console.log('🔍 [getPostList] 开始构建查询');
05  201
06  202    // 获取被屏蔽的用户ID列表（使用缓存）
07  203    let blockedUserIds = [];
08  204    try {
09  205      const getBlockedUserIds = require('../_lib/get-blocked-user-ids');
10  206      blockedUserIds = await getBlockedUserIds(openid, db);
11  207      console.log('🔍 [getPostList] 被屏蔽的用户数量:', blockedUserIds.length);
12  208    } catch (blockError) {
13  209      console.error('❌ [getPostList] 获取屏蔽列表失败:', blockError);
14  210    }
15  211
16  212    let query = db.collection('posts').aggregate();
17  213
18  214    // 构建筛选条件
19  215    const matchConditions = { isHidden: _.neq(true) };
20  216
21  217    if (isActivityQuery) {
22  218      matchConditions.$or = [
23  219        {
24  220          isActivityPost: true,
25  221          activityId: safeActivityId
26  222        },
27  223        {
28  224          joinedActivityId: safeActivityId
29  225        }
30  226      ];
31  227    } else {
32  228      // 常规列表默认排除活动帖子
33  229      matchConditions.isActivityPost = _.neq(true);
34  230    }
35  231
36  232    // 如果指定了 isPoem 参数，添加诗歌筛选条件
37  233    if (isPoem !== undefined) {
38  234      if (isPoem === true) {
39  235        // 只获取诗歌：isPoem 必须为 true
40  236        matchConditions.isPoem = true;
41  237      } else {
42  238        // 只获取非诗歌：isPoem 为 false 或不存在（$ne: true 会匹配 false、null 和不存在的字段）
43  239        matchConditions.isPoem = _.neq(true);
44  240      }
45  241      console.log('🔍 [getPostList] 添加isPoem筛选条件:', isPoem);
46  242    }
47  243
48  244    // 如果指定了 isOriginal 参数，添加原创筛选条件
49  245    if (isOriginal !== undefined) {
50  246      if (isOriginal === true) {
```

```
01  247         // 只获取原创：isOriginal 必须为 true
02  248         matchConditions.isOriginal = true;
03  249       } else {
04  250         // 只获取非原创：isOriginal 为 false 或不存在（$ne: true 会匹配 false、null 和不存在的字
05  ↳ 段）
06  251         matchConditions.isOriginal = _.neq(true);
07  252       }
08  253       console.log('🔍 [getPostList] 添加isOriginal筛选条件:', isOriginal);
09  254     }
10  255
11  256     // 如果指定了 tag 参数，添加标签筛选条件
12  257     if (tag) {
13  258       matchConditions.tags = tag;  // 匹配包含该标签的文档
14  259       matchConditions['tags.0'] = { $exists: true };  // 确保 tags 数组至少有一个元素
15  260       console.log('🔍 [getPostList] 添加tag筛选条件:', tag);
16  261     }
17  262
18  263     // 如果指定了 isDiscussion 参数，添加讨论筛选条件
19  264     if (isDiscussion !== undefined) {
20  265       if (isDiscussion === true) {
21  266         // 只获取讨论：isDiscussion 必须为 true
22  267         matchConditions.isDiscussion = true;
23  268       } else {
24  269         // 只获取非讨论：isDiscussion 为 false 或不存在（$ne: true 会匹配 false、null 和不存在的
25  ↳ 字段）
26  270         matchConditions.isDiscussion = _.neq(true);
27  271       }
28  272       console.log('🔍 [getPostList] 添加isDiscussion筛选条件:', isDiscussion);
29  273     }
30  274
31  275     // 按诗人（作者）筛选
32  276     if (author && author.trim()) {
33  277       matchConditions.author = author.trim();
34  278       console.log('🔍 [getPostList] 添加author筛选条件:', author);
35  279     }
36  280
37  281     // 过滤被屏蔽用户的帖子（查询阶段先过滤 _openid，结果处理阶段再过滤 realAuthorOpenid）
38  282     if (blockedUserIds.length > 0) {
39  283       // 在查询阶段过滤普通帖子的 _openid
40  284       matchConditions._openid = _.nin(blockedUserIds);
41  285       console.log('🔍 [getPostList] 添加屏蔽用户过滤条件（_openid），被屏蔽用户数:', blockedUser
42  ↳ Ids.length);
43  286     }
44  287
45  288     console.log('🔍 [getPostList] 最终筛选条件:', JSON.stringify(matchConditions, null, 2));
46  289
47  290     // 如果有筛选条件，应用 match
48  291     if (Object.keys(matchConditions).length > 0) {
49  292       query = query.match(matchConditions);
50  293     }
```

```
01  294
02  295     if (isActivityQuery) {
03  296       query = query.addFields({
04  297         activityOrder: {
05  298           $cond: [
06  299             { $eq: ['$isActivityPost', true] },
07  300             0,
08  301             1
09  302           ]
10  303         },
11  304         activitySortTime: {
12  305           $cond: [
13  306             { $eq: ['$isActivityPost', true] },
14  307             { $ifNull: ['$activityPublishTime', '$createTime'] },
15  308             { $ifNull: ['$joinedActivityAt', '$createTime'] }
16  309           ]
17  310         }
18  311       });
19  312     }
20  313
21  314     // 判断是否启用随机混合逻辑
22  315     // 只有当没有筛选条件时（广场页面），才启用随机混合
23  316     // 有筛选条件时，只返回时间顺序的帖子
24  317     const hasFilter = isPoem !== undefined || isOriginal !== undefined || isDiscussion !== undef
25      ↳ined || tag || author || includeActivity === true || !!safeActivityId;
26  318     const enableRandomMix = !hasFilter;  // 没有筛选条件时启用随机混合
27  319
28  320     let posts = [];
29  321     let timeOrderedPosts = []; // 在外层声明，便于在日志输出时使用
30  322     let randomPosts = []; // 在外层声明，便于在日志输出时使用
31  323
32  324     if (enableRandomMix) {
33  325       // 广场页面：使用随机混合逻辑（6个时间顺序 + 4个随机）
34  326       const TIME_ORDERED_COUNT = 6;  // 按时间顺序的帖子数量
35  327       const RANDOM_COUNT = 4;        // 随机帖子的数量
36  328
37  329       // 1. 先获取按时间顺序的帖子（6个）
38  330       const timeOrderedQuery = query.sort(
39  331         isActivityQuery
40  332           ? { activityOrder: 1, activitySortTime: -1, createTime: -1 }
41  333           : { createTime: -1 }
42  334       )
43  335         .skip(skip)
44  336         .limit(TIME_ORDERED_COUNT);
45  337
46  338       const timeOrderedRes = await timeOrderedQuery.end();
47  339       timeOrderedPosts = timeOrderedRes.list || []; // 使用外层声明的变量
48  340
49  341       // 2. 获取随机帖子（4个）
50  342       randomPosts = []; // 重置随机帖子数组
```

```
01  343     const timeOrderedPostIds = timeOrderedPosts.map(p => p._id).filter(Boolean);
02  344
03  345     // 即使没有时间顺序的帖子，也尝试获取随机帖子
04  346     try {
05  347       // 构建随机帖子查询条件（排除已获取的时间顺序帖子）
06  348       const randomPostsMatchConditions = {
07  349         isHidden: _.neq(true),
08  350         isActivityPost: _.neq(true)
09  351       };
10  352
11  353       // 复制所有筛选条件（虽然这里理论上没有筛选条件，但为了保险仍保留）
12  354       // 如果有时间顺序的帖子，排除它们
13  355       if (timeOrderedPostIds.length > 0) {
14  356         randomPostsMatchConditions._id = _.nin(timeOrderedPostIds);
15  357       }
16  358
17  359       // 排除被屏蔽的用户
18  360       if (blockedUserIds.length > 0) {
19  361         randomPostsMatchConditions._openid = _.nin(blockedUserIds);
20  362       }
21  363
22  364       // 先统计符合条件的帖子总数
23  365       const countRes = await db.collection('posts').aggregate()
24  366         .match(randomPostsMatchConditions)
25  367         .count('total')
26  368         .end();
27  369       const totalPosts = (countRes.list && countRes.list[0] && countRes.list[0].total) || 0;
28  370
29  371       if (totalPosts > 0) {
30  372         // 改进的随机策略：多次随机查询以增加随机性
31  373         let candidatePosts = [];
32  374         const targetCandidates = Math.min(totalPosts, RANDOM_COUNT * 5); // 获取更多候选以增加
33    ↳ 随机性
34  375
35  376         // 如果总数较少，直接获取全部
36  377         if (totalPosts <= targetCandidates) {
37  378           const allQuery = db.collection('posts').aggregate()
38  379             .match(randomPostsMatchConditions)
39  380             .limit(totalPosts);
40  381           const allRes = await allQuery.end();
41  382           candidatePosts = allRes.list || [];
42  383         } else {
43  384           // 多次随机查询，每次从不同位置获取
44  385           const queriesPerBatch = 3; // 分批查询
45  386           const perBatchCount = Math.ceil(targetCandidates / queriesPerBatch);
46  387
47  388           for (let i = 0; i < queriesPerBatch && candidatePosts.length < targetCandidates; i++
48    ↳ ) {
49  389             // 每次随机选择一个 skip 位置
50  390             const maxSkip = Math.max(0, totalPosts - perBatchCount);
```

```
01  391             const randomSkip = Math.floor(Math.random() * (maxSkip + 1));
02  392
03  393             const randomQuery = db.collection('posts').aggregate()
04  394               .match(randomPostsMatchConditions)
05  395               .sort({ createTime: -1 })  // 虽然按时间排序，但 skip 是随机的
06  396               .skip(randomSkip)
07  397               .limit(perBatchCount);
08  398
09  399             const randomRes = await randomQuery.end();
10  400             const batchPosts = randomRes.list || [];
11  401
12  402             // 添加到候选列表，并去重
13  403             const existingIds = new Set(candidatePosts.map(p => p._id));
14  404             const newPosts = batchPosts.filter(p => !existingIds.has(p._id));
15  405             candidatePosts = candidatePosts.concat(newPosts);
16  406           }
17  407         }
18  408
19  409         // 如果候选帖子还不够，从所有帖子中随机选择 skip 位置再获取
20  410         if (candidatePosts.length < RANDOM_COUNT && totalPosts > candidatePosts.length) {
21  411           const remainingNeeded = RANDOM_COUNT - candidatePosts.length;
22  412           const existingIds = new Set(candidatePosts.map(p => p._id));
23  413
24  414           // 再尝试几次随机查询
25  415           for (let attempt = 0; attempt < 5 && candidatePosts.length < targetCandidates; attem
26    ↳ pt++) {
27  416             const maxSkip = Math.max(0, totalPosts - remainingNeeded * 2);
28  417             const randomSkip = Math.floor(Math.random() * (maxSkip + 1));
29  418
30  419             const randomQuery = db.collection('posts').aggregate()
31  420               .match(randomPostsMatchConditions)
32  421               .sort({ createTime: -1 })
33  422               .skip(randomSkip)
34  423               .limit(remainingNeeded * 2);
35  424
36  425             const randomRes = await randomQuery.end();
37  426             const batchPosts = randomRes.list || [];
38  427             const newPosts = batchPosts.filter(p => !existingIds.has(p._id));
39  428             candidatePosts = candidatePosts.concat(newPosts);
40  429             newPosts.forEach(p => existingIds.add(p._id));
41  430           }
42  431         }
43  432
44  433         // 从候选帖子中彻底随机打乱并选择（使用 Fisher-Yates shuffle 算法）
45  434         const shuffled = candidatePosts.slice();
46  435         for (let i = shuffled.length - 1; i > 0; i--) {
47  436           const j = Math.floor(Math.random() * (i + 1));
48  437           const temp = shuffled[i];
49  438           shuffled[i] = shuffled[j];
50  439           shuffled[j] = temp;
```

```
01  440         }
02  441         randomPosts = shuffled.slice(0, Math.min(RANDOM_COUNT, shuffled.length));
03  442       }
04  443     } catch (randomError) {
05  444       console.error('❌ [getPostList] 获取随机帖子失败:', randomError);
06  445     }
07  446
08  447     // 3. 混合帖子：真正随机混合时间顺序和随机帖子
09  448     if (randomPosts.length > 0) {
10  449       // 创建所有帖子的合并列表
11  450       const allPosts = timeOrderedPosts.concat(randomPosts);
12  451
13  452       // 使用 Fisher-Yates shuffle 算法真正随机打乱
14  453       for (let i = allPosts.length - 1; i > 0; i--) {
15  454         const j = Math.floor(Math.random() * (i + 1));
16  455         const temp = allPosts[i];
17  456         allPosts[i] = allPosts[j];
18  457         allPosts[j] = temp;
19  458       }
20  459
21  460       posts = allPosts;
22  461     } else {
23  462       posts = timeOrderedPosts.slice();
24  463     }
25  464   } else {
26  465     // 有筛选条件时：只返回时间顺序的帖子
27  466     const timeOrderedQuery = query.sort(
28  467       isActivityQuery
29  468         ? { activityOrder: 1, activitySortTime: -1, createTime: -1 }
30  469         : { createTime: -1 }
31  470     )
32  471       .skip(skip)
33  472       .limit(limit);
34  473
35  474     const timeOrderedRes = await timeOrderedQuery.end();
36  475     posts = timeOrderedRes.list || [];
37  476   }
38  477
39  478   // 确保返回的帖子数量不超过 limit
40  479   posts = posts.slice(0, limit);
41  480
42  481   // 批量查询点赞状态
43  482   let voterMap = new Set();
44  483   if (posts.length > 0) {
45  484     try {
46  485       const postIds = posts.map(post => post._id);
47  486       const voteRes = await db.collection('votes_log')
48  487         .where({
49  488           _openid: openid,
50  489           type: 'post',
```

```
01  490            postId: _.in(postIds)
02  491          })
03  492          .field({ postId: true })
04  493          .get();
05  494        voterMap = new Set(voteRes.data.map(item => item.postId));
06  495      } catch (voteError) {
07  496        console.error('❌ [getPostList] 批量查询点赞记录失败:', voteError);
08  497      }
09  498    }
10  499
11  500    // 处理帖子数据，并再次过滤被屏蔽用户（双重保险）
12  501    let processedPosts = posts.map(post => {
13  502      const authorName = post.authorName || post.authorNameSnapshot || '匿名用户';
14  503      const authorAvatar = post.authorAvatar || post.authorAvatarSnapshot || '';
15  504      const authorSignature = post.authorSignature || ''; // 签名URL（匿名帖子可能为空）
16  505      const commentCount = post.commentCount || 0;
17  506      const isVoted = voterMap.has(post._id);
18  507
19  508      // 组诗字段兜底：确保列表返回至少前三段
20  509      const seriesBlocksRaw = Array.isArray(post.seriesBlocks) ? post.seriesBlocks : [];
21  510      const seriesBlocks = seriesBlocksRaw.slice(0, 3).map((b, idx) => ({
22  511        id: b.id || `series-${idx}`,
23  512        subtitle: b.subtitle || b.subTitle || '',
24  513        content: b.content || '',
25  514        highlightSentence: b.highlightSentence || (b.content ? String(b.content).split(/\r?\n/).
26    ↳ find(l => l && l.trim()) || '' : ''),
27  515        highlightLines: Array.isArray(b.highlightLines) ? b.highlightLines : []
28  516      }));
29  517      const seriesBlockCount = post.seriesBlockCount || seriesBlocksRaw.length || seriesBlocks.l
30    ↳ ength;
31  518      const isSeries = post.isSeries || seriesBlocks.length > 0;
32  519
33  520      return {
34  521        ...post,
35  522        isSeries,
36  523        seriesBlocks,
37  524        seriesBlockCount,
38  525        authorName,
39  526        authorAvatar,
40  527        authorSignature,
41  528        commentCount,
42  529        isVoted,
43  530        tags: Array.isArray(post.tags) ? post.tags : []
44  531      };
45  532    });
46  533
47  534    // 双重保险：前端再次过滤被屏蔽用户的帖子（包括匿名帖子的 realAuthorOpenid）
48  535    if (blockedUserIds.length > 0) {
49  536      processedPosts = processedPosts.filter(post => {
50  537        // 检查普通帖子的 _openid
```

```
01  538         if (blockedUserIds.includes(post._openid)) {
02  539           return false;
03  540         }
04  541         // 检查匿名帖子的 realAuthorOpenid
05  542         if (post.realAuthorOpenid && blockedUserIds.includes(post.realAuthorOpenid)) {
06  543           return false;
07  544         }
08  545         return true;
09  546       });
10  547     }
11  548
12  549     // --- 优化图片URL转换逻辑 ---
13  550     const fileIDs = new Set(); // 使用 Set 避免重复 fileID
14  551
15  552     processedPosts.forEach(post => {
16  553       // 保证 imageUrls、originalImageUrls 一定为数组
17  554       if (!Array.isArray(post.imageUrls)) post.imageUrls = post.imageUrls ? [post.imageUrls] : [
18  ↳ ];
19  555       if (!Array.isArray(post.originalImageUrls)) post.originalImageUrls = post.originalImageUrl
20  ↳ s ? [post.originalImageUrls] : [];
21  556
22  557       // 收集所有需要转换的 fileID
23  558       const urlsToCheck = [
24  559         ...post.imageUrls,
25  560         ...post.originalImageUrls,
26  561         post.imageUrl,
27  562         post.originalImageUrl,
28  563         post.authorAvatar,
29  564         post.authorSignature, // 添加签名URL
30  565         post.poemBgImage
31  566       ].filter(url => url && url.startsWith('cloud://'));
32  567
33  568       urlsToCheck.forEach(url => fileIDs.add(url));
34  569     });
35  570
36  571     if (fileIDs.size > 0) {
37  572       try {
38  573         const fileListResult = await cloud.getTempFileURL({ fileList: Array.from(fileIDs) });
39  574         const urlMap = new Map();
40  575
41  576         fileListResult.fileList.forEach(item => {
42  577           if (item.status === 0) {
43  578             urlMap.set(item.fileID, item.tempFileURL);
44  579           }
45  580         });
46  581
47  582         // 批量转换所有帖子的图片URL
48  583         processedPosts.forEach(post => {
49  584           const convertUrl = (url) => urlMap.get(url) || url;
50  585
```

```
01  586         if (post.imageUrl) post.imageUrl = convertUrl(post.imageUrl);
02  587         if (post.originalImageUrl) post.originalImageUrl = convertUrl(post.originalImageUrl);
03  588         if (post.authorAvatar) post.authorAvatar = convertUrl(post.authorAvatar);
04  589         if (post.authorSignature) post.authorSignature = convertUrl(post.authorSignature); //
05    ↳ 转换签名URL
06  590         if (post.poemBgImage) post.poemBgImage = convertUrl(post.poemBgImage);
07  591
08  592         if (Array.isArray(post.imageUrls)) {
09  593           post.imageUrls = post.imageUrls.map(convertUrl);
10  594         }
11  595         if (Array.isArray(post.originalImageUrls)) {
12  596           post.originalImageUrls = post.originalImageUrls.map(convertUrl);
13  597         }
14  598       });
15  599     } catch (fileError) {
16  600       console.error('❌ [getPostList] 图片URL转换失败:', fileError);
17  601     }
18  602   }
19  603
20  604   console.log('✅ [getPostList] ========== 云函数执行完成 ==========');
21  605   console.log('✅ [getPostList] 返回帖子数量:', processedPosts.length);
22  606   if (processedPosts.length > 0) {
23  607     console.log('✅ [getPostList] 前3个帖子时间:', processedPosts.slice(0, 3).map(p => ({
24  608       id: p._id,
25  609       createTime: p.createTime
26  610     })));
27  611   }
28  612
29  613   return {
30  614     success: true,
31  615     posts: processedPosts
32  616   };
33  617
34  618 } catch (e) {
35  619   console.error('❌ [getPostList] 云函数执行失败:', e);
36  620   console.error('❌ [getPostList] 错误详情:', {
37  621     message: e.message,
38  622     stack: e.stack,
39  623     name: e.name
40  624   });
41  625   return {
42  626     success: false,
43  627     error: {
44  628       message: e.message,
45  629       stack: e.stack
46  630     }
47  631   };
48  632 }
49  633 };
50  FILE FILE 14 | functions/getPostDetail/index.js | 帖子详情云函数
```

```
01  1  // 云函数入口文件
02  2  const cloud = require('wx-server-sdk');
03  3
04  4  cloud.init({
05  5    env: cloud.DYNAMIC_CURRENT_ENV
06  6  });
07  7
08  8  const db = cloud.database();
09  9
10 10  // 云函数入口函数
11 11  exports.main = async (event, context) => {
12 12    const wxContext = cloud.getWXContext();
13 13    const wxCtxOpenid = wxContext.OPENID;
14 14    const eventOpenid = event.openid;
15 15    const openid = eventOpenid || wxCtxOpenid;
16 16    const { postId } = event;
17 17
18 18    if (!openid) {
19 19      return {
20 20        success: false,
21 21        message: '无法获取用户 openid，请重新登录',
22 22        code: 'NO_OPENID'
23 23      };
24 24    }
25 25    console.log('🔍 [getPostDetail] openid来源:', {
26 26      eventOpenid: eventOpenid ? '提供' : '未提供',
27 27      wxCtxOpenid: wxCtxOpenid ? '提供' : '未提供',
28 28      chosenOpenidSource: eventOpenid ? 'event.openid' : 'wxContext.OPENID',
29 29      chosenOpenidExists: !!openid
30 30    });
31 31
32 32    if (!postId) {
33 33      return {
34 34        success: false,
35 35        message: 'Post ID is required.'
36 36      };
37 37    }
38 38
39 39    try {
40 40      // 1. 根据 postId 获取帖子详情
41 41      const postRes = await db.collection('posts').doc(postId).get();
42 42      const post = postRes.data;
43 43      // 如果帖子被隐藏且当前用户不是作者，禁止访问
44 44      if (post && post.isHidden === true && post._openid !== openid) {
45 45        return {
46 46          success: false,
47 47          code: 'POST_HIDDEN',
48 48          message: '该内容已隐藏'
49 49        };
50 50      }
```

```
01  51
02  52    if (!post) {
03  53      return {
04  54        success: false,
05  55        message: 'Post not found.'
06  56      };
07  57    }
08  58
09  59    console.log('获取到的帖子数据:', {
10  60      _id: post._id,
11  61      title: post.title,
12  62      commentCount: post.commentCount,
13  63      votes: post.votes
14  64    });
15  65
16  66    // 2. 根据帖子的 _openid 获取作者信息
17  67    const userRes = await db.collection('users').where({
18  68      _openid: post._openid
19  69    }).get();
20  70
21  71    // If author is not found, provide a default object for robustness
22  72    const author = userRes.data[0] || {
23  73        nickName: '匿名用户',
24  74        avatarUrl: ''
25  75    };
26  76
27  77    // 3. 获取当前用户的点赞记录
28  78    const voteRes = await db.collection('votes_log').where({
29  79        _openid: openid,
30  80        postId: postId,
31  81        type: 'post'
32  82    }).get();
33  83
34  84    // 4. 组合最终结果
35  85    const resultPost = {
36  86      ...post,
37  87      authorName: post.authorName || post.authorNameSnapshot || author.nickName || '匿名用户',
38  88      authorAvatar: post.authorAvatar || post.authorAvatarSnapshot || author.avatarUrl || '',
39  89      authorSignature: post.authorSignature || '', // 签名URL（优先使用帖子中存储的值）
40  90      isAuthor: post._openid === openid,
41  91      isVoted: voteRes.data.length > 0,
42  92      tags: post.tags || [] // 确保标签字段存在
43  93    };
44  94    // 保证 imageUrls、originalImageUrls 一定为数组
45  95    if (!Array.isArray(resultPost.imageUrls)) resultPost.imageUrls = resultPost.imageUrls ? [res
46  ↳ultPost.imageUrls] : [];
47  96    if (!Array.isArray(resultPost.originalImageUrls)) resultPost.originalImageUrls = resultPost.
48  ↳originalImageUrls ? [resultPost.originalImageUrls] : [];
49  97
50  98    // --- Efficiently convert FileIDs to temp URLs ---
```

```
01      99    const fileIDs = [];
02     100    if (resultPost.imageUrl && resultPost.imageUrl.startsWith('cloud://')) {
03     101      fileIDs.push(resultPost.imageUrl);
04     102    }
05     103    if (resultPost.imageUrls && Array.isArray(resultPost.imageUrls)) {
06     104      resultPost.imageUrls.forEach(url => {
07     105        if (url && url.startsWith('cloud://')) {
08     106          fileIDs.push(url);
09     107        }
10     108      });
11     109    }
12     110    if (resultPost.originalImageUrl && resultPost.originalImageUrl.startsWith('cloud://')) {
13     111      fileIDs.push(resultPost.originalImageUrl);
14     112    }
15     113    if (resultPost.originalImageUrls && Array.isArray(resultPost.originalImageUrls)) {
16     114      resultPost.originalImageUrls.forEach(url => {
17     115        if (url && url.startsWith('cloud://')) {
18     116          fileIDs.push(url);
19     117        }
20     118      });
21     119    }
22     120    if (resultPost.authorAvatar && resultPost.authorAvatar.startsWith('cloud://')) {
23     121      fileIDs.push(resultPost.authorAvatar);
24     122    }
25     123    if (resultPost.authorSignature && resultPost.authorSignature.startsWith('cloud://')) {
26     124      fileIDs.push(resultPost.authorSignature);
27     125    }
28     126
29     127    if (fileIDs.length > 0) {
30     128      const fileListResult = await cloud.getTempFileURL({ fileList: fileIDs });
31     129      const urlMap = new Map();
32     130      fileListResult.fileList.forEach(item => {
33     131        if (item.status === 0) {
34     132          urlMap.set(item.fileID, item.tempFileURL);
35     133        }
36     134      });
37     135
38     136      if (resultPost.imageUrl && urlMap.has(resultPost.imageUrl)) {
39     137        resultPost.imageUrl = urlMap.get(resultPost.imageUrl);
40     138      }
41     139      if (resultPost.imageUrls && Array.isArray(resultPost.imageUrls)) {
42     140        resultPost.imageUrls = resultPost.imageUrls.map(url =>
43     141          url && urlMap.has(url) ? urlMap.get(url) : url
44     142        );
45     143      }
46     144      if (resultPost.originalImageUrl && urlMap.has(resultPost.originalImageUrl)) {
47     145        resultPost.originalImageUrl = urlMap.get(resultPost.originalImageUrl);
48     146      }
49     147      if (resultPost.originalImageUrls && Array.isArray(resultPost.originalImageUrls)) {
50     148        resultPost.originalImageUrls = resultPost.originalImageUrls.map(url =>
```

```
01  149          url && urlMap.has(url) ? urlMap.get(url) : url
02  150        );
03  151      }
04  152      if (resultPost.authorAvatar && urlMap.has(resultPost.authorAvatar)) {
05  153        resultPost.authorAvatar = urlMap.get(resultPost.authorAvatar);
06  154      }
07  155      if (resultPost.authorSignature && urlMap.has(resultPost.authorSignature)) {
08  156        resultPost.authorSignature = urlMap.get(resultPost.authorSignature);
09  157      }
10  158    }
11  159
12  160    // 如果帖子没有commentCount字段，则实时计算评论数量
13  161    let commentCount = resultPost.commentCount;
14  162    console.log('从数据库获取的commentCount:', commentCount, '类型:', typeof commentCount);
15  163
16  164    if (commentCount === undefined || commentCount === null) {
17  165      console.log('需要实时计算评论数量');
18  166      const commentsRes = await db.collection('comments').where({
19  167        postId: postId
20  168      }).count();
21  169      commentCount = commentsRes.total;
22  170      console.log('实时计算的评论数量:', commentCount);
23  171
24  172      // 同时更新帖子数据库中的commentCount字段，避免下次再计算
25  173      try {
26  174        await db.collection('posts').doc(postId).update({
27  175          data: {
28  176            commentCount: commentCount
29  177          }
30  178        });
31  179        console.log('已更新数据库中的commentCount字段');
32  180      } catch (updateError) {
33  181        console.error('更新commentCount字段失败:', updateError);
34  182      }
35  183    } else {
36  184      console.log('使用数据库中的commentCount:', commentCount);
37  185    }
38  186
39  187    console.log('最终返回的评论数量:', commentCount);
40  188
41  189    return {
42  190      post: resultPost,
43  191      commentCount: commentCount, // 添加评论数量
44  192      success: true
45  193    };
46  194
47  195  } catch (e) {
48  196    console.error(e);
49  197    return {
50  198      success: false,
```

01	199	error: e
02	200	};
03	201	}
04	202	};
05	FILE	**FILE 15 | functions/vote/index.js | 点赞云函数**
06	1	const cloud = require('wx-server-sdk')
07	2	cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV })
08	3	
09	4	const db = cloud.database()
10	5	const _ = db.command
11	6	
12	7	// buckets for growth stats on user profile
13	8	// 统一阈值：与前端显示一致 (1-3/4-7/8-15/16+)
14	9	const BUCKETS = [
15	10	{ key: 'seed', min: 1, max: 3 },
16	11	{ key: 'leaf', min: 4, max: 7 },
17	12	{ key: 'flower', min: 8, max: 15 },
18	13	{ key: 'peach', min: 16, max: Infinity },
19	14	]
20	15	const bucketOf = (v) => {
21	16	v = typeof v === 'number' ? v : 0
22	17	if (v < 1) return null
23	18	for (const b of BUCKETS) {
24	19	if (v >= b.min && v <= b.max) return b.key
25	20	}
26	21	return null
27	22	}
28	23	
29	24	async function updateAuthorGrowthCounts(authorOpenid, oldVotes, newVotes) {
30	25	const from = bucketOf(oldVotes)
31	26	const to = bucketOf(newVotes)
32	27	if (from === to) return
33	28	const data = { growthUpdatedAt: db.serverDate() }
34	29	if (from) data[`growthCounts.${from}`] = _.inc(-1)
35	30	if (to) data[`growthCounts.${to}`] = _.inc(1)
36	31	const upd = await db.collection('users').where({ _openid: authorOpenid }).update({ data })
37	32	if (!upd.stats || upd.stats.updated === 0) {
38	33	try {
39	34	await db.collection('users').add({
40	35	data: {
41	36	_openid: authorOpenid,
42	37	growthCounts: {
43	38	seed: to === 'seed' ? 1 : 0,
44	39	leaf: to === 'leaf' ? 1 : 0,
45	40	flower: to === 'flower' ? 1 : 0,
46	41	peach: to === 'peach' ? 1 : 0,
47	42	},
48	43	growthUpdatedAt: db.serverDate(),
49	44	createTime: new Date(),
50	45	updateTime: new Date(),

```
01  46        },
02  47      })
03  48    } catch (e) {
04  49      // ignore race
05  50    }
06  51  }
07  52}
08  53
09  54// 云函数入口函数
10  55exports.main = async (event, context) => {
11  56
12  57  try {
13  58    const { postId } = event
14  59    const wxContext = cloud.getWXContext()
15  60    const wxCtxOpenid = wxContext.OPENID
16  61    const eventOpenid = event.openid
17  62    const openid = eventOpenid || wxCtxOpenid
18  63
19  64    // 读取帖子，拿作者与当前票数（用于分段迁移）
20  65    const postSnapBefore = await db.collection('posts').doc(postId).get()
21  66    const postBefore = postSnapBefore.data
22  67    if (!postBefore) {
23  68      return { success: false, message: 'POST_NOT_FOUND' }
24  69    }
25  70    const authorOpenid = postBefore._openid
26  71    const oldVotes = postBefore.votes || 0
27  72
28  73    if (!openid) {
29  74      console.error('❌ [vote] 无法获取用户 openid');
30  75      return {
31  76        success: false,
32  77        message: '无法获取用户 openid，请重新登录',
33  78        code: 'NO_OPENID'
34  79      }
35  80    }
36  81
37  82    // 1. 查找 votes_log 表，精确查找 type 为 'post' 的记录
38  83    const log = await db.collection('votes_log').where({
39  84      _openid: openid,
40  85      postId: postId,
41  86      type: 'post'
42  87    }).get()
43  88
44  89
45  90    let updatedPost;
46  91    let isLiked = false;
47  92
48  93    if (log.data.length > 0) {
49  94      // 2. 如果找到了记录，说明是"取消点赞"
50  95      await db.collection('votes_log').doc(log.data[0]._id).remove()
```

```
01  96      await db.collection('posts').doc(postId).update({ data: { votes: _.inc(-1) } })
02  97      const postSnapAfter = await db.collection('posts').doc(postId).get()
03  98      const newVotes = (postSnapAfter.data && postSnapAfter.data.votes) || (oldVotes - 1)
04  99      await updateAuthorGrowthCounts(authorOpenid, oldVotes, newVotes)
05  100     isLiked = false
06  101   } else {
07  102     // 3. 如果没找到记录，说明是"点赞"
08  103     await db.collection('votes_log').add({
09  104       data: {
10  105         _openid: openid,
11  106         postId: postId,
12  107         type: 'post',
13  108         createTime: new Date()
14  109       }
15  110     })
16  111     await db.collection('posts').doc(postId).update({ data: { votes: _.inc(1) } })
17  112     const postSnapAfter = await db.collection('posts').doc(postId).get()
18  113     const newVotes = (postSnapAfter.data && postSnapAfter.data.votes) || (oldVotes + 1)
19  114     await updateAuthorGrowthCounts(authorOpenid, oldVotes, newVotes)
20  115     isLiked = true
21  116
22  117     // === 新增：创建点赞消息通知 ===
23  118     try {
24  119       // 获取帖子信息
25  120       const postResult = await db.collection('posts').doc(postId).get()
26  121       const post = postResult.data
27  122
28  123       // 获取点赞者信息
29  124       const userResult = await db.collection('users').where({
30  125         _openid: openid
31  126       }).limit(1).get()
32  127       const user = userResult.data[0]
33  128
34  129       // 如果给自己点赞，不发送通知
35  130       if (post._openid !== openid) {
36  131         // 根据帖子实际字段确定内容类型
37  132         let contentType = 'post';
38  133         let contentTypeText = '帖子';
39  134
40  135         if (post.isDiscussion) {
41  136           contentType = 'discussion';
42  137           contentTypeText = '讨论';
43  138         } else if (post.isPoem) {
44  139           if (post.isOriginal) {
45  140             contentType = 'original';
46  141             contentTypeText = '原创诗歌';
47  142           } else {
48  143             contentType = 'non-original';
49  144             contentTypeText = '诗歌';
50  145           }
```

```
01  146         }
02  147
03  148         await db.collection('messages').add({
04  149           data: {
05  150             fromUserId: openid,
06  151             fromUserName: user ? user.nickName : '微信用户',
07  152             fromUserAvatar: user ? user.avatarUrl : '',
08  153             toUserId: post._openid,
09  154             type: 'like',
10  155             postId: postId,
11  156             postTitle: post.title || '无标题',
12  157             contentType: contentType,
13  158             content: `${user ? user.nickName : '微信用户'} 点赞了你的${contentTypeText}`,
14  159             isRead: false,
15  160             createTime: new Date()
16  161           }
17  162         })
18  163       }
19  164     } catch (msgError) {
20  165       console.error('创建点赞消息失败:', msgError)
21  166       // 不影响主流程，只是记录错误
22  167     }
23  168   }
24  169
25  170   // 4. 无论点赞还是取消，都重新获取文章的最新数据
26  171   updatedPost = await db.collection('posts').doc(postId).get();
27  172
28  173   const finalVotes = updatedPost.data.votes || 0;
29  174
30  175   const result = {
31  176     success: true,
32  177     votes: finalVotes, // 返回最新的点赞数
33  178     isLiked: isLiked
34  179   };
35  180
36  181   return result;
37  182
38  183 } catch (e) {
39  184   console.error('云函数执行失败:', e);
40  185   return {
41  186     success: false,
42  187     error: {
43  188       message: e.message,
44  189       stack: e.stack
45  190     }
46  191   }
47  192 }
48  193 }
49  FILE FILE 16 | functions/addComment/index.js | 评论云函数
50  1   const cloud = require('wx-server-sdk');
```

```
01  2
02  3  cloud.init({
03  4    env: cloud.DYNAMIC_CURRENT_ENV
04  5  });
05  6
06  7  const db = cloud.database();
07  8  const _ = db.command;
08  9
09  10 function uniqueCloudFileIds(values = []) {
10  11   return [...new Set(
11  12     values.filter((value) => typeof value === 'string' && value.startsWith('cloud://'))
12  13   )];
13  14 }
14  15
15  16 async function buildTempUrlMap(fileIds = []) {
16  17   const validFileIds = uniqueCloudFileIds(fileIds);
17  18   if (!validFileIds.length) {
18  19     return new Map();
19  20   }
20  21
21  22   try {
22  23     const result = await cloud.getTempFileURL({ fileList: validFileIds });
23  24     const urlMap = new Map();
24  25     (result.fileList || []).forEach((item) => {
25  26       if (item && item.status === 0 && item.fileID && item.tempFileURL) {
26  27         urlMap.set(item.fileID, item.tempFileURL);
27  28       }
28  29     });
29  30     return urlMap;
30  31   } catch (error) {
31  32     console.error('buildTempUrlMap error', error);
32  33     return new Map();
33  34   }
34  35 }
35  36
36  37 function mapTempUrls(values = [], urlMap = new Map()) {
37  38   if (!Array.isArray(values) || values.length === 0) {
38  39     return [];
39  40   }
40  41   return values.map((value) => urlMap.get(value) || value);
41  42 }
42  43
43  44 exports.main = async (event, context) => {
44  45   const wxContext = cloud.getWXContext();
45  46   const openid = wxContext.OPENID || event.openid;
46  47
47  48   if (!openid) {
48  49     return {
49  50       success: false,
50  51       message: 'Failed to get user openid, please login again.',
```

```
01  52      code: 'NO_OPENID'
02  53    };
03  54  }
04  55
05  56  const {
06  57    postId,
07  58    content = '',
08  59    parentId,
09  60    replyToAuthorName,
10  61    imageUrls = [],
11  62    originalImageUrls = [],
12  63    isAnonymous = false
13  64  } = event;
14  65
15  66  const trimmedContent = (content || '').trim();
16  67  const sanitizedImageUrls = Array.isArray(imageUrls) ? imageUrls.filter(Boolean) : [];
17  68  let sanitizedOriginalImageUrls = Array.isArray(originalImageUrls) ? originalImageUrls.filter(B
18    ↳ oolean) : [];
19  69
20  70  if (sanitizedOriginalImageUrls.length === 0 && sanitizedImageUrls.length > 0) {
21  71    sanitizedOriginalImageUrls = sanitizedImageUrls.slice();
22  72  }
23  73
24  74  const hasContent = trimmedContent.length > 0;
25  75  const hasImages = sanitizedImageUrls.length > 0;
26  76
27  77  if (!postId) {
28  78    return { success: false, message: 'Post ID is required.' };
29  79  }
30  80  if (!hasContent && !hasImages) {
31  81    return { success: false, message: 'Comment content or images are required.' };
32  82  }
33  83
34  84  try {
35  85    const userResult = await db.collection('users').where({
36  86      _openid: openid
37  87    }).limit(1).get();
38  88    const user = userResult.data[0];
39  89    const createTime = new Date();
40  90
41  91    const commentData = {
42  92      _openid: openid,
43  93      postId,
44  94      content: trimmedContent,
45  95      likes: 0,
46  96      createTime,
47  97      imageUrls: sanitizedImageUrls,
48  98      originalImageUrls: sanitizedOriginalImageUrls,
49  99      hasImages,
50 100      isAnonymous,
```

```
      authorName: isAnonymous ? '匿名用户' : (user ? user.nickName : '微信用户'),
      authorAvatar: isAnonymous ? '/static/images/avatar.png' : (user ? user.avatarUrl : ''),
      realAuthorOpenid: isAnonymous ? openid : null
    };

    if (parentId) {
      commentData.parentId = parentId;
      if (replyToAuthorName) {
        commentData.replyToAuthorName = replyToAuthorName;
      }
    }

    const addResult = await db.collection('comments').add({
      data: commentData
    });

    await db.collection('posts').doc(postId).update({
      data: {
        commentCount: _.inc(1)
      }
    });

    try {
      const postResult = await db.collection('posts').doc(postId).get();
      const post = postResult.data;

      let notifyUserId = null;
      let shouldNotify = false;

      if (parentId) {
        try {
          const parentCommentResult = await db.collection('comments').doc(parentId).get();
          const parentComment = parentCommentResult.data;

          if (parentComment) {
            const parentCommentAuthorId = parentComment.isAnonymous
              ? (parentComment.realAuthorOpenid || parentComment._openid)
              : parentComment._openid;

            if (parentCommentAuthorId !== openid && !isAnonymous) {
              notifyUserId = parentCommentAuthorId;
              shouldNotify = true;
            }
          }
        } catch (commentError) {
          console.error('query parent comment failed', commentError);
        }
      } else if (post && post._openid !== openid && !isAnonymous) {
        notifyUserId = post._openid;
        shouldNotify = true;
```

```
01  151     }
02  152
03  153     if (shouldNotify && notifyUserId) {
04  154       let contentType = 'post';
05  155       let contentTypeText = '帖子';
06  156
07  157       if (post && post.isDiscussion) {
08  158         contentType = 'discussion';
09  159         contentTypeText = '讨论';
10  160       } else if (post && post.isPoem) {
11  161         if (post.isOriginal) {
12  162           contentType = 'original';
13  163           contentTypeText = '原创诗歌';
14  164         } else {
15  165           contentType = 'non-original';
16  166           contentTypeText = '诗歌';
17  167         }
18  168       }
19  169
20  170       const senderName = user ? user.nickName : '微信用户';
21  171       const messageContent = parentId
22  172         ? `${senderName} 回复了你的评论`
23  173         : `${senderName} 评论了你的${contentTypeText}`;
24  174
25  175       await db.collection('messages').add({
26  176         data: {
27  177           fromUserId: openid,
28  178           fromUserName: senderName,
29  179           fromUserAvatar: user ? user.avatarUrl : '',
30  180           toUserId: notifyUserId,
31  181           type: 'comment',
32  182           postId,
33  183           postTitle: post && post.title ? post.title : '无标题',
34  184           contentType,
35  185           isReply: !!parentId,
36  186           content: messageContent,
37  187           commentId: addResult._id,
38  188           isRead: false,
39  189           createTime: new Date()
40  190         }
41  191       });
42  192     }
43  193   } catch (msgError) {
44  194     console.error('create comment message failed', msgError);
45  195   }
46  196
47  197   const returnComment = {
48  198     ...commentData,
49  199     _id: addResult._id,
50  200     parentId: parentId || null,
```

```
01  201       replyToAuthorName: replyToAuthorName || null,
02  202       liked: false,
03  203       canDelete: true,
04  204       replies: []
05  205     };
06  206
07  207     const urlMap = await buildTempUrlMap([
08  208       returnComment.authorAvatar,
09  209       ...returnComment.imageUrls,
10  210       ...returnComment.originalImageUrls
11  211     ]);
12  212
13  213     if (urlMap.size > 0) {
14  214       if (returnComment.authorAvatar) {
15  215         returnComment.authorAvatar = urlMap.get(returnComment.authorAvatar) || returnComment.aut
16      ↳horAvatar;
17  216       }
18  217       returnComment.imageUrls = mapTempUrls(returnComment.imageUrls, urlMap);
19  218       returnComment.originalImageUrls = mapTempUrls(returnComment.originalImageUrls, urlMap);
20  219     }
21  220
22  221     return {
23  222       success: true,
24  223       message: 'Comment added successfully.',
25  224       commentId: addResult._id,
26  225       comment: returnComment
27  226     };
28  227   } catch (error) {
29  228     console.error('addComment error', error);
30  229     return {
31  230       success: false,
32  231       message: 'Failed to add comment.',
33  232       error: error.toString()
34  233     };
35  234   }
36  235 };
37
38
39
40
41
42
43
44
45
46
47
48
49
50
```